Ministério da Educação
Secretaria de Educação Profissional e Tecnológica
Instituto Federal Catarinense
Campus Videira

VINÍCIUS SLONGO

SISTEMA DE SUPORTE À VALIDAÇÃO DE PLANOS DE ENSINO COM MODELOS DE LINGUAGEM: UM ESTUDO DE CASO NO CURSO DE CIÊNCIA DA COMPUTAÇÃO DO IFC – CAMPUS VIDEIRA

Videira
2026

VINÍCIUS SLONGO

SISTEMA DE SUPORTE À VALIDAÇÃO DE PLANOS DE ENSINO COM MODELOS DE LINGUAGEM: UM ESTUDO DE CASO NO CURSO DE CIÊNCIA DA COMPUTAÇÃO DO IFC – CAMPUS VIDEIRA

Trabalho de Conclusão de Curso apresentado ao Curso de graduação em Ciência da Computação do Instituto Federal Catarinense – Campus Videira para obtenção do título de bacharel em Ciência da Computação.
Orientador: Prof. Me. Fabricio Bizotto

Videira
2026

VINÍCIUS SLONGO

SISTEMA DE SUPORTE À VALIDAÇÃO DE PLANOS DE ENSINO COM MODELOS DE LINGUAGEM: UM ESTUDO DE CASO NO CURSO DE CIÊNCIA DA COMPUTAÇÃO DO IFC – CAMPUS VIDEIRA

Este Trabalho de Curso foi julgado adequado para a obtenção do título de Bacharel em Ciência da Computação e aprovado em sua forma final pelo curso de graduação em Ciência da Computação do Instituto Federal Catarinense – Campus Videira.

Videira (SC), _____ de _____ de 2026

Prof. Me. Fabricio Bizotto
Instituto Federal Catarinense – Campus Videira

BANCA EXAMINADORA

Prof.(ª) Titulação Nome do(a) avaliador(a) 1
Instituto Federal Catarinense – Campus Videira

Prof.(ª) Titulação Nome do(a) avaliador(a) 2
Instituto Federal Catarinense – Campus Videira

Dedico este trabalho à minha família, que sempre acreditou na educação como caminho.

AGRADECIMENTOS

Agradeço, em primeiro lugar, à minha família, pelo apoio e pela paciência ao longo de toda a graduação.

Ao professor Fabricio Bizotto, pela orientação, pela disponibilidade e pelas sugestões que deram forma aos critérios de avaliação adotados neste trabalho.

Aos professores e servidores do Instituto Federal Catarinense – Campus Videira, pela formação recebida e pelo acesso aos planos de ensino que tornaram possível a avaliação do sistema.

Aos colegas de curso, pela convivência e pela troca de conhecimento durante esses anos.

Ensinar não é transferir conhecimento, mas criar as possibilidades para a sua própria produção ou a sua construção.

Paulo Freire, *Pedagogia da autonomia*

RESUMO

No Instituto Federal Catarinense (IFC), cada componente curricular possui um plano de ensino que deve conter os elementos exigidos pela Organização Didática da instituição, e a conferência desses documentos é feita manualmente pela coordenação e pelo Núcleo Docente Estruturante (NDE), tarefa repetitiva e sujeita a inconsistências entre avaliadores. Este trabalho apresenta o desenvolvimento e a avaliação de um sistema semiautomático de apoio à validação de planos de ensino do curso de Ciência da Computação do IFC – Campus Videira. A solução, construída sobre a plataforma de automação n8n e executada em contêineres Docker, obtém os planos em PDF de uma pasta local, extrai e segmenta o texto em seções, avalia doze critérios com um modelo de linguagem de grande escala (LLM) aberto, o Qwen2.5 de 7 bilhões de parâmetros executado localmente pelo Ollama, e registra o resultado de cada critério (OK, INCOMPLETO ou AUSENTE) em arquivos JSON consultados por um painel *web*, que também os exporta em CSV. Para reduzir alucinações, a arquitetura combina o LLM com saída estruturada por esquema JSON e com regras determinísticas, como a contagem de referências bibliográficas por expressões regulares. O desenvolvimento seguiu um processo iterativo e incremental, e a avaliação usou os 29 planos de ensino do semestre 2024/1. Ao longo das iterações, a proporção de planos com falha técnica caiu de 86,2% para zero, e a versão final processou o lote completo em 8 minutos e 54 segundos, com média de 11,5 segundos de inferência por plano em uma GPU de 6 GB. Na execução final, 11 planos (38%) foram considerados conformes, 3 (10%) pendentes e 15 (52%) reprovados, e a ausência do horário de atendimento ao estudante foi a não conformidade mais frequente. No critério verificável por oráculo estrutural, o sistema obteve *recall* de 100%, precisão de 93,8% e Kappa de Cohen de 0,93. A análise de erros identificou limitações, como textos provisórios do sistema acadêmico aceitos como conteúdo válido e observações gerais do modelo contraditórias com o resultado por critério, que orientam os trabalhos futuros. Conclui-se que o sistema é viável como ferramenta de triagem, capaz de indicar ao avaliador os planos e os itens que exigem revisão, desde que a decisão final permaneça com ele; o efeito dessa triagem sobre o tempo de conferência não foi medido e depende de avaliação com usuários.

Palavras-chave: Inteligência artificial. Modelos de linguagem. Plano de ensino. Automação de processos. n8n.

ABSTRACT

At the Instituto Federal Catarinense (IFC), every course unit has a teaching plan that must contain the elements required by the institution's academic regulations, and these documents are checked manually by the program coordination and the Structuring Teaching Nucleus (NDE), a repetitive task prone to inconsistencies between reviewers. This work presents the development and evaluation of a semi-automatic system that supports the validation of teaching plans in the Computer Science program at IFC – Videira Campus. The solution, built on the n8n automation platform and deployed with Docker containers, reads the plans as PDF files from a local folder, extracts and segments their text into sections, assesses twelve criteria with an open large language model (LLM), the 7-billion-parameter Qwen2.5 running locally on Ollama, and stores the result of each criterion (OK, INCOMPLETE or MISSING) in JSON files displayed by a web dashboard, which also exports them as CSV. To reduce hallucinations, the architecture combines the LLM with JSON-schema structured output and with deterministic rules, such as counting bibliographic references with regular expressions. Development followed an iterative and incremental process, and the evaluation used the 29 teaching plans of the 2024/1 semester. Across iterations, the share of plans with technical failures dropped from 86.2% to zero, and the final version processed the whole batch in 8 minutes and 54 seconds, averaging 11.5 seconds of inference per plan on a 6 GB GPU. In the final run, 11 plans (38%) were compliant, 3 (10%) pending and 15 (52%) rejected, and the missing student office hours was the most frequent non-compliance. On the criterion that could be checked by a structural oracle, the system reached 100% recall, 93.8% precision and a Cohen's Kappa of 0.93. The error analysis revealed limitations, such as placeholder texts from the academic system being accepted as valid content and general remarks from the model contradicting the per-criterion results, which guide future work. The system proved viable as a screening tool that points the reviewer to the plans and items that need revision, provided that the final decision remains with the reviewer; its effect on review time was not measured and requires a user study.

Keywords: Artificial intelligence. Language models. Teaching plan. Process automation. n8n.

LISTA DE ILUSTRAÇÕES

Figura 1 – Artigos publicados por ano sobre IA e educação (2006–2022) 27
Figura 2 – Arquitetura do sistema 36
Figura 3 – Modelo conceitual dos dados 42
Figura 4 – Estrutura do fluxo de validação no n8n 44
Figura 5 – Diagrama de casos de uso 44
Figura 6 – Diagrama de sequência da validação 45
Figura 7 – Diagrama de atividades da interpretação da resposta 46
Figura 8 – Diagrama de estados de um plano de ensino 47
Figura 9 – *Dashboard* de validação com o plano P05 expandido 51
Figura 10 – Proporção de planos com status ERRO por data de execução 52
Figura 11 – Velocidade de geração por configuração 55
Figura 12 – Resultado por critério na execução final (29 planos) 56

LISTA DE TABELAS

Tabela 1 – Comparação entre os trabalhos relacionados e este trabalho 29
Tabela 2 – Critérios de avaliação e regras de conformidade 32
Tabela 3 – Ambiente experimental 33
Tabela 4 – Iterações de desenvolvimento 37
Tabela 5 – Requisitos funcionais 38
Tabela 6 – Requisitos não funcionais 39
Tabela 7 – Atores do sistema 39
Tabela 8 – Histórias de usuário 41
Tabela 9 – Estrutura do registro de resultado de cada plano 42
Tabela 10 – Causas de falha identificadas e correções aplicadas 53
Tabela 11 – Estatísticas de inferência por plano na execução final (n = 29) 54
Tabela 12 – Matriz de confusão do critério horário de atendimento 57
Tabela 13 – Verificação dos apontamentos da execução final (exceto horário de atendimento) 58

LISTA DE ABREVIATURAS E SIGLAS

API	*Application Programming Interface* (Interface de Programação de Aplicações)
CCO	Curso de Ciência da Computação
CPU	*Central Processing Unit* (Unidade Central de Processamento)
FN	Falso negativo
FP	Falso positivo
GPU	*Graphics Processing Unit* (Unidade de Processamento Gráfico)
HTML	*HyperText Markup Language*
HTTP	*Hypertext Transfer Protocol*
IA	Inteligência Artificial
IFC	Instituto Federal Catarinense
JSON	*JavaScript Object Notation*
KV	*Key-Value* (cache de chaves e valores da atenção)
LDB	Lei de Diretrizes e Bases da Educação Nacional
LGPD	Lei Geral de Proteção de Dados Pessoais
LLM	*Large Language Model* (Modelo de Linguagem de Grande Escala)
MLOps	*Machine Learning Operations*
NDE	Núcleo Docente Estruturante
OAuth	*Open Authorization*
PDF	*Portable Document Format*
PLN	Processamento de Linguagem Natural
PPC	Projeto Pedagógico de Curso
RAG	*Retrieval-Augmented Generation* (Geração Aumentada por Recuperação)
RF	Requisito Funcional
RN	Regra de Negócio

RNF	Requisito Não Funcional
SIGAA	Sistema Integrado de Gestão de Atividades Acadêmicas
UML	*Unified Modeling Language*
VN	Verdadeiro negativo
VP	Verdadeiro positivo
VRAM	*Video Random Access Memory*

SUMÁRIO

1 INTRODUÇÃO 15
1.1 Problema de pesquisa 16
1.2 Objetivos 16
1.2.1 Objetivo Geral 16
1.2.2 Objetivos Específicos 17
1.3 Justificativa 17
1.4 Estrutura do trabalho 18
2 FUNDAMENTAÇÃO TEÓRICA 19
2.1 O plano de ensino e as normas institucionais 19
2.2 Modelos de linguagem 20
2.2.1 A arquitetura *Transformer* 20
2.2.2 Limitações dos LLMs 21
2.3 Técnicas para respostas confiáveis 21
2.3.1 Engenharia de *prompt* e contextualização normativa 22
2.3.2 Geração aumentada por recuperação 22
2.3.3 Geração estruturada 23
2.3.4 LLMs como avaliadores e combinação com regras determinísticas 23
2.4 Execução local de modelos de linguagem 24
2.5 Automação de fluxos e infraestrutura 24
2.6 IA centrada no ser humano 25
2.7 Métricas de avaliação 26
2.8 Trabalhos relacionados 27
3 METODOLOGIA 30
3.1 Classificação da pesquisa 30
3.2 Etapas da pesquisa 30
3.3 Corpus 31
3.4 Critérios de avaliação 31
3.5 Ambiente experimental 32
3.6 Protocolo de avaliação 33
3.7 Aspectos éticos 34

4 DESENVOLVIMENTO 35
4.1 Apresentação do Sistema 35
4.2 Metodologia de desenvolvimento de software 36
4.3 Requisitos Funcionais 37
4.4 Requisitos Não Funcionais 38
4.5 Atores do Sistema 39
4.6 Regras de Negócio 40
4.7 Histórias de Usuário 40
4.8 Modelagem do Banco de Dados 41
4.9 Diagramas 43
4.9.1 Diagramas Estruturais 43
4.9.2 Diagramas Comportamentais 43
4.10 Implementação 47
4.10.1 Infraestrutura 47
4.10.2 Aquisição e extração de texto 48
4.10.3 Segmentação e contagem de referências 48
4.10.4 Construção do *prompt* e saída estruturada 48
4.10.5 Chamada ao modelo e interpretação da resposta 49
4.10.6 *Dashboard* 49
4.11 Prototipação 50
4.12 Testes 50
5 AVALIAÇÃO E RESULTADOS 52
5.1 Robustez técnica ao longo das iterações 52
5.2 Desempenho computacional 54
5.3 Resultados da validação do corpus 55
5.4 Concordância com o oráculo estrutural 56
5.5 Análise de erros 57
5.6 Discussão 59
6 CONCLUSÃO 61
6.1 Limitações 62
6.2 Trabalhos futuros 62
REFERÊNCIAS 64

APÊNDICE A ***Prompt*** **e esquema JSON enviados ao modelo 69**
APÊNDICE B Segmentação do texto e interpretação da resposta 73
APÊNDICE C Artefatos para a análise dos resultados 78

1 INTRODUÇÃO

Os avanços recentes em Inteligência Artificial (IA), em especial no Processamento de Linguagem Natural (PLN) e nos modelos de linguagem de grande escala (*Large Language Models*, LLMs), ampliaram as possibilidades de automação de tarefas que antes dependiam exclusivamente da leitura humana de documentos (Zhao *et al.*, 2023; Adiguzel; Kaya; Cansu, 2023). A UNESCO aponta que a IA pode apoiar tanto a aprendizagem quanto a gestão educacional, desde que sua adoção seja acompanhada de políticas que garantam equidade e supervisão humana (Pedró *et al.*, 2019). No Brasil, pesquisa da Chegg.org divulgada pela CNN Brasil (2023) indica que cerca de metade dos universitários já utiliza ferramentas de IA em sua rotina acadêmica, o que evidencia a rápida difusão dessas tecnologias no ambiente educacional.

No campo da gestão educacional, Sposato (2025) classifica as aplicações de IA na liderança e na administração de instituições de ensino e destaca a automação de tarefas rotineiras, a análise de documentos e o suporte à tomada de decisão como usos recorrentes. A revisão sistemática de Tan, Cheng e Ling (2025) reforça esse cenário ao identificar o apoio ao planejamento docente e a otimização de tarefas administrativas entre as aplicações mais frequentes, ao mesmo tempo em que aponta a escassez de estudos sobre processos administrativos do ensino superior. Walter (2024), por sua vez, ressalta que a integração da IA na educação exige letramento em IA e pensamento crítico, o que reforça a necessidade de manter a supervisão humana em qualquer solução automatizada.

Entre os documentos que estruturam o trabalho docente está o plano de ensino. A Lei de Diretrizes e Bases da Educação Nacional (LDB) atribui aos docentes a incumbência de elaborar e cumprir seu plano de trabalho e determina que as instituições informem aos estudantes, antes de cada período letivo, os programas dos componentes curriculares e seus critérios de avaliação (Brasil, 1996). No Instituto Federal Catarinense (IFC), cada componente curricular possui um plano de ensino elaborado pelo docente a cada semestre, e a Organização Didática da instituição define os elementos que esse documento deve conter, como ementa, objetivos, conteúdos, metodologia, avaliação e referências bibliográficas (Instituto Federal Catarinense, 2022). A verificação de que cada plano contém esses elementos é feita manualmente pela coordenação de curso e pelo Núcleo Docente Estruturante (NDE), lendo documento por documento.

Essa conferência manual é repetitiva e sujeita a inconsistências entre avaliadores, e seu custo cresce com o número de componentes curriculares. Automatizá-la, contudo, não

é trivial: os planos de ensino do IFC são emitidos pelo Sistema Integrado de Gestão de Atividades Acadêmicas (SIGAA) em um formato padronizado, com as mesmas seções em todos os documentos, mas o conteúdo de cada seção é texto livre e heterogêneo, escrito por docentes diferentes. Regras fixas conseguem verificar se uma seção existe ou quantas referências ela contém, mas não se uma metodologia descreve de fato como as aulas serão conduzidas. Esse tipo de julgamento exige interpretar o texto, tarefa para a qual os LLMs são adequados. Esses modelos, porém, estão sujeitos a alucinações, isto é, à geração de conteúdo plausível porém incorreto (Ji *et al.*, 2023), de modo que, aplicados à conferência dos planos, seus apontamentos precisam ser verificáveis e a decisão sobre cada plano precisa permanecer com o avaliador humano (United States of America, 2023; Shneiderman, 2022).

Diante desse cenário, este trabalho propõe desenvolver um sistema semiautomático de apoio à validação dos planos de ensino do curso de Ciência da Computação do IFC – Campus Videira e avaliá-lo com os planos reais de um semestre letivo. O sistema deverá obter os planos em PDF de uma pasta compartilhada, extrair e segmentar seu texto, avaliar doze critérios com um LLM aberto executado localmente, combinar essa avaliação com regras determinísticas e apresentar o resultado de cada critério em um painel *web*, para que a coordenação e o NDE possam identificar os planos e os itens que exigem revisão. A avaliação busca verificar se essa abordagem é tecnicamente viável em um equipamento local e em que medida seus apontamentos concordam com verificações de referência.

1.1 PROBLEMA DE PESQUISA

De que forma um sistema baseado em modelos de linguagem pode apoiar a verificação da conformidade dos planos de ensino às normas do IFC, indicando de forma confiável os itens que exigem revisão e preservando o controle humano sobre a decisão final?

1.2 OBJETIVOS

Para responder ao problema de pesquisa, foram definidos um objetivo geral e quatro objetivos específicos, apresentados a seguir.

1.2.1 Objetivo Geral

Desenvolver e avaliar um sistema semiautomático baseado em IA para apoiar a validação dos planos de ensino do curso de Ciência da Computação do IFC – Campus Videira,

alinhado à Organização Didática da instituição.

1.2.2 Objetivos Específicos

- Identificar os elementos obrigatórios dos planos de ensino a partir da Organização Didática do IFC e das orientações da coordenação do curso, convertendo-os em critérios de avaliação verificáveis.
- Projetar e implementar um *pipeline* de automação que extraia e segmente o texto dos planos, avalie cada critério com um LLM executado localmente e registre os resultados de forma rastreável.
- Disponibilizar um painel *web* que permita acompanhar os planos processados e consultar o resultado e a justificativa de cada critério.
- Avaliar o sistema com os 29 planos de ensino reais do semestre 2024/1, medindo a proporção de planos com falha técnica em cada iteração (robustez técnica), o tempo de inferência por plano e de processamento do lote (desempenho computacional) e a concordância dos resultados com um oráculo estrutural e com a conferência manual dos apontamentos no texto dos planos, e analisar os erros encontrados para propor aprimoramentos.

1.3 JUSTIFICATIVA

A conferência manual dos planos de ensino consome tempo da coordenação e do NDE que poderia ser dedicado a ações pedagógicas de maior impacto, como a análise da coerência entre componentes curriculares e a atualização do Projeto Pedagógico de Curso (PPC). Por ser repetitiva, essa conferência também está sujeita a variações entre avaliadores e entre semestres, o que dificulta garantir que todos os planos atendam aos mesmos critérios.

Do ponto de vista científico, a revisão de Vieira e Moura (2025) sobre as experiências com IA nos Institutos Federais constatou pouca produção acadêmica sobre o tema e identificou a IA como apoio administrativo como a forma de incorporação menos explorada. A aplicação de LLMs à verificação normativa de documentos acadêmicos internos é, portanto, um nicho pouco investigado, especialmente em instituições públicas brasileiras e com modelos executados localmente.

Do ponto de vista prático, a opção por ferramentas de código aberto e por um modelo executado em equipamento próprio elimina custos de licenciamento e evita o envio dos documentos a serviços externos de IA, o que facilita a adoção institucional e o atendimento à

Lei Geral de Proteção de Dados Pessoais (LGPD) (Brasil, 2018). Por fim, a abordagem semiautomática, em que o sistema aponta possíveis não conformidades e o avaliador humano decide, segue as recomendações de uso responsável de IA na educação (United States of America, 2023; Pedró *et al.*, 2019).

1.4 ESTRUTURA DO TRABALHO

Além desta introdução, o trabalho está organizado em cinco capítulos. O Capítulo 2 apresenta a fundamentação teórica sobre planos de ensino, modelos de linguagem, técnicas para reduzir alucinações, ferramentas de automação e métricas de avaliação, além dos trabalhos relacionados. O Capítulo 3 descreve a metodologia da pesquisa, incluindo o corpus utilizado e o protocolo de avaliação. O Capítulo 4 documenta o desenvolvimento do sistema: requisitos, atores, regras de negócio, histórias de usuário, modelagem de dados, diagramas e protótipo. O Capítulo 5 apresenta e discute os resultados da avaliação. Por fim, o Capítulo 6 apresenta as conclusões, as limitações e as sugestões de trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos que sustentam o trabalho. A Seção 2.1 discute o plano de ensino e as normas que regem sua elaboração no IFC. As Seções 2.2 a 2.4 tratam dos modelos de linguagem, de suas limitações, das técnicas empregadas para torná-los confiáveis e de sua execução em equipamento local. A Seção 2.5 aborda as ferramentas de automação e infraestrutura, a Seção 2.6 discute o papel do avaliador humano e a Seção 2.7 apresenta as métricas de avaliação utilizadas. Por fim, a Seção 2.8 revisa os trabalhos relacionados.

2.1 O PLANO DE ENSINO E AS NORMAS INSTITUCIONAIS

O planejamento é uma atividade de reflexão e de organização da ação docente, por meio da qual o professor articula os objetivos, os conteúdos, os métodos e a avaliação de forma coerente com a proposta pedagógica da instituição (Libâneo, 1994). O plano de ensino é o registro desse planejamento para um componente curricular em um período letivo. Ele serve, ao mesmo tempo, como guia para o docente, como compromisso público com os estudantes e como instrumento de acompanhamento para a gestão do curso.

A LDB estabelece que os docentes devem elaborar e cumprir seu plano de trabalho segundo a proposta pedagógica do estabelecimento e que as instituições de educação superior devem informar aos interessados, antes de cada período letivo, os programas dos cursos e componentes curriculares, seus requisitos e seus critérios de avaliação (Brasil, 1996). No IFC, a Organização Didática dos Cursos detalha essas exigências e define os elementos que devem constar no plano de ensino, que é submetido à apreciação da coordenação e do NDE do curso (Instituto Federal Catarinense, 2022).

Na prática do IFC, o plano de ensino é registrado no SIGAA e exportado como um documento PDF com estrutura padronizada: identificação do componente e do docente, ementa, metodologia, procedimentos de avaliação, horário de atendimento, objetivos, conteúdo programático, cronograma de aulas, avaliações previstas e referências básicas e complementares. Embora a ordem e os rótulos das seções sejam fixos, o conteúdo de cada seção é texto livre, escrito por docentes diferentes, com estilos, extensões e graus de detalhamento distintos. Essa combinação de estrutura fixa e conteúdo livre é o que motiva a abordagem híbrida adotada neste trabalho: regras determinísticas para o que é estrutural e modelos de linguagem para o que exige interpretação.

2.2 MODELOS DE LINGUAGEM

Como visto na seção anterior, o plano de ensino tem estrutura fixa, mas o conteúdo de cada seção é texto livre. Verificar se esse conteúdo atende às normas exige uma tecnologia capaz de interpretar linguagem natural, e a que fundamenta este trabalho são os modelos de linguagem. Eles são objeto de estudo do PLN, a área da IA que trata da compreensão e da geração de linguagem humana por computadores (Russell; Norvig, 2021; Jurafsky; Martin, 2025).

Um modelo de linguagem atribui probabilidades a sequências de palavras e, de forma equivalente, estima a probabilidade da próxima unidade de texto (*token*) dada a sequência anterior (Jurafsky; Martin, 2025). Os LLMs são modelos de linguagem com bilhões de parâmetros, treinados sobre grandes volumes de texto. Com o aumento de escala, esses modelos passaram a exibir a capacidade de executar tarefas novas apenas a partir de instruções e exemplos fornecidos na entrada, sem ajuste dos pesos, fenômeno conhecido como aprendizado em contexto (*in-context learning*) (Brown *et al.*, 2020; Zhao *et al.*, 2023). É essa capacidade que permite usar um mesmo modelo, sem treinamento adicional, para avaliar se uma ementa ou uma metodologia atende a uma regra descrita em linguagem natural.

2.2.1 A arquitetura *Transformer*

A capacidade de executar tarefas a partir de instruções, descrita acima, só se tornou possível com o aumento de escala dos modelos, e esse aumento, por sua vez, foi viabilizado por uma arquitetura específica. Os LLMs atuais são baseados na arquitetura *Transformer*, proposta por Vaswani *et al.* (2017), que substitui as redes recorrentes por um mecanismo de atenção que, para cada *token*, calcula o quanto cada um dos demais *tokens* da sequência é relevante para sua representação. Na formulação original, a atenção é dada pela Equação 2.1, em que Q, K e V são matrizes de consultas, chaves e valores obtidas por projeções lineares da entrada e d_k é a dimensão das chaves (Vaswani *et al.*, 2017).

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2.1}$$

Como a atenção relaciona diretamente quaisquer duas posições da sequência, o *Transformer* captura dependências de longa distância e permite paralelizar o treinamento, o que viabilizou o crescimento dos modelos para bilhões de parâmetros. Os modelos generativos, como os da família GPT e o Qwen utilizado neste trabalho, empregam apenas a pilha decodificadora e geram texto de forma autorregressiva, um *token* por vez (Brown *et al.*, 2020; Qwen

Team, 2024). Durante a geração, as chaves e os valores já calculados são armazenados em um *cache* (*KV cache*) para não serem recomputados, e seu tamanho cresce linearmente com o comprimento do contexto, o que torna o contexto máximo um fator determinante do consumo de memória.

2.2.2 Limitações dos LLMs

A mesma arquitetura que permite aos LLMs gerar texto fluente, um *token* por vez, não garante que esse texto seja correto. Apesar da fluência, os LLMs apresentam limitações relevantes para aplicações de verificação. A principal é a alucinação, definida como a geração de conteúdo sem sentido ou infiel à fonte fornecida (Ji *et al.*, 2023). Em um verificador de documentos, uma alucinação pode se manifestar como a afirmação de que uma seção está ausente quando ela existe, ou o contrário. Outra limitação é a sensibilidade à posição da informação: Liu *et al.* (2024) mostraram que o desempenho de LLMs em contextos longos cai quando a informação relevante está no meio da entrada, o que favorece entradas curtas e bem delimitadas. Os modelos também têm dificuldade com operações exatas, como contagens, e não têm acesso a normas institucionais específicas que não estejam presentes em seus dados de treinamento.

2.3 TÉCNICAS PARA RESPOSTAS CONFIÁVEIS

As limitações descritas na seção anterior não desaparecem com a simples escolha de um modelo maior. Por isso, foram propostas técnicas que atuam em torno do modelo, sem alterar seus pesos, para tornar suas respostas mais confiáveis. Essas técnicas interferem em três pontos do processo. Na entrada, definem o que o modelo recebe: instruções bem estruturadas e o conhecimento necessário para a tarefa, como fazem a engenharia de *prompt* e a geração aumentada por recuperação. Na saída, restringem o formato da resposta para que ela possa ser processada por software, como faz a geração estruturada. Na verificação, conferem o julgamento do modelo ou o substituem por regras exatas quando a tarefa permite. Como nenhuma delas exige treinar novamente o modelo, todas podem ser aplicadas a um modelo aberto executado localmente. As subseções a seguir descrevem cada técnica e a forma como foi aproveitada neste trabalho.

2.3.1 Engenharia de *prompt* e contextualização normativa

A técnica mais direta atua sobre as instruções fornecidas ao modelo. A engenharia de *prompt* consiste em estruturar as instruções fornecidas ao modelo de forma a obter respostas no formato e com a qualidade desejados. White *et al.* (2023) catalogam padrões de *prompt*, entre eles a definição de uma persona ("você é um avaliador de planos de ensino"), a especificação de um modelo de saída e a imposição de restrições explícitas. No contexto deste trabalho, a estratégia central é a contextualização normativa: as regras de cada critério, derivadas da Organização Didática e das orientações da coordenação, são escritas no próprio *prompt*, junto com o texto de cada seção do plano, de modo que o modelo julgue o documento com base nas regras fornecidas e não em seu conhecimento genérico.

2.3.2 Geração aumentada por recuperação

Na contextualização normativa, o conhecimento que o modelo deve usar é escrito diretamente no *prompt*. Quando esse conhecimento é extenso demais para isso, ele pode ser buscado automaticamente. É o que faz a Geração Aumentada por Recuperação (*Retrieval-Augmented Generation*, RAG), proposta por Lewis *et al.* (2020), combina um modelo gerador com um mecanismo de busca: antes de responder, o sistema recupera de uma base externa, normalmente indexada por vetores de *embeddings*, os trechos mais relevantes para a pergunta e os fornece ao modelo como contexto. A técnica reduz alucinações e permite usar conhecimento atualizado ou específico de uma organização sem retreinar o modelo (Lewis *et al.*, 2020; Gao *et al.*, 2023).

A RAG é especialmente vantajosa quando a base de conhecimento é grande demais para caber no contexto do modelo. No problema tratado aqui, o conhecimento normativo necessário para avaliar um plano se resume a doze regras curtas, que ocupam poucas centenas de *tokens*. Nesse cenário, incluir todas as regras diretamente no *prompt* produz o mesmo efeito de fundamentação da RAG, com menos componentes e sem o risco de a etapa de recuperação deixar de trazer uma regra relevante. Por isso, o sistema desenvolvido adota a contextualização normativa direta, e a RAG com base vetorial é apontada como evolução para quando o sistema precisar consultar documentos extensos, como o PPC completo (Seção 6.2).

2.3.3 Geração estruturada

As duas técnicas anteriores atuam sobre a entrada do modelo; a geração estruturada atua sobre sua saída. Para que a resposta de um LLM seja processada por software, ela precisa seguir um formato previsível. Pedir "responda em JSON" no *prompt* não garante isso: o modelo pode produzir JSON malformado, omitir campos ou usar valores fora do domínio esperado. A geração guiada (*guided* ou *constrained generation*) resolve o problema restringindo, a cada passo da decodificação, os *tokens* admissíveis àqueles que mantêm a saída válida segundo uma gramática ou expressão regular (Willard; Louf, 2023). O Ollama implementa essa técnica a partir de um JSON Schema informado no parâmetro `format` da requisição (Ollama, 2024; JSON Schema, 2022). Com isso, campos obrigatórios e enumerações, como os valores OK, INCOMPLETO e AUSENTE, passam a ser garantidos pela decodificação e não apenas solicitados ao modelo.

2.3.4 LLMs como avaliadores e combinação com regras determinísticas

Mesmo com uma entrada bem fundamentada e uma saída em formato válido, resta saber se o julgamento do modelo está correto, o que corresponde ao terceiro ponto de atuação, a verificação. Usar um LLM para julgar a qualidade de um texto é uma prática estudada na literatura. Zheng *et al.* (2023) mostraram que LLMs fortes, quando usados como juízes, alcançam concordância com preferências humanas comparável à concordância entre humanos, embora apresentem vieses, como a preferência por respostas mais longas. Gilardi, Alizadeh e Kubli (2023) verificaram que um LLM superou anotadores de plataformas de *crowdsourcing* em tarefas de anotação de texto. Ambos os estudos, contudo, reforçam a necessidade de validar as saídas do modelo contra referências humanas, tema também enfatizado na revisão de Chang *et al.* (2024) sobre avaliação de LLMs.

Uma consequência prática dessas limitações é atribuir a cada técnica o tipo de tarefa em que ela é mais confiável. Verificações exatas e estruturais, como contar referências ou detectar uma seção vazia, são resolvidas de forma determinística por código; julgamentos que exigem interpretação, como decidir se uma metodologia descreve como as aulas serão conduzidas, ficam a cargo do LLM. Essa divisão de trabalho é um dos princípios de projeto do sistema descrito no Capítulo 4.

2.4 EXECUÇÃO LOCAL DE MODELOS DE LINGUAGEM

As técnicas da seção anterior pressupõem um modelo disponível para receber as requisições, que pode ser acessado por um serviço externo ou executado pela própria instituição. Este trabalho adota a segunda opção. Executar um LLM em equipamento próprio evita custos por requisição e mantém os documentos dentro da instituição, mas impõe restrições de memória. Um modelo de 7 bilhões de parâmetros em precisão de 16 bits ocupa cerca de 14 GB apenas com os pesos, acima da memória de vídeo (VRAM) de GPUs de uso pessoal. A quantização reduz esse consumo ao representar os pesos com menos bits, com perda de qualidade pequena quando bem aplicada (Dettmers *et al.*, 2022). O projeto llama.cpp popularizou o formato GGUF e esquemas de quantização como o Q4_K_M, que armazena a maior parte dos pesos com cerca de 4 bits (ggml-org, 2026). O Ollama, utilizado neste trabalho, empacota esse motor de inferência e expõe os modelos por meio de uma API HTTP local (Ollama, 2026).

O modelo escolhido foi o Qwen2.5 de 7,6 bilhões de parâmetros, um modelo de pesos abertos, distribuído sob a licença Apache 2.0, com suporte a contexto longo, a múltiplos idiomas, incluindo o português, e ênfase declarada na geração de saídas estruturadas como JSON (Qwen Team, 2024). Na versão quantizada Q4_K_M, o modelo ocupa cerca de 4,7 GB, o que permite executá-lo em uma GPU de 6 GB, ainda que com pouca margem para o *KV cache*.

Duas otimizações de inferência foram relevantes neste trabalho. O *FlashAttention* reorganiza o cálculo da atenção para reduzir os acessos à memória da GPU, tornando-o mais rápido e econômico sem alterar o resultado (Dao *et al.*, 2022). A quantização do *KV cache* para 8 bits reduz a memória ocupada pelo contexto, permitindo que uma fração maior do modelo permaneça na GPU. O efeito combinado dessas configurações sobre o desempenho do sistema é medido no Capítulo 5.

2.5 AUTOMAÇÃO DE FLUXOS E INFRAESTRUTURA

O modelo de linguagem, por si só, não forma um sistema: é preciso obter os documentos, extrair seu texto, enviá-lo ao modelo, registrar as respostas e executar tudo isso de forma reprodutível. Esta seção apresenta as ferramentas usadas para essa orquestração. O n8n é uma plataforma de automação de fluxos de trabalho de código-fonte disponível, na qual um fluxo (*workflow*) é composto visualmente por nós conectados. Cada nó executa uma operação, como acionar o fluxo em um horário, ler arquivos de um serviço de armazenamento, fazer uma requisição HTTP ou executar um trecho de código JavaScript, e entrega uma lista de itens ao nó

seguinte (n8n, 2026). A plataforma oferece nós para ler e gravar arquivos em disco, integrações prontas com serviços externos, como Google Drive e Google Sheets, e permite expor fluxos como *webhooks*, isto é, como endereços HTTP que executam o fluxo ao serem acessados. Essas características permitem construir um *pipeline* completo de processamento de documentos sem desenvolver uma aplicação do zero, e o histórico de execuções mantido pela plataforma facilita a depuração e a auditoria.

O Docker permite empacotar uma aplicação e suas dependências em contêineres isolados e reprodutíveis, que se comportam da mesma forma em qualquer máquina com o Docker instalado (Merkel, 2014). O Docker Compose descreve, em um único arquivo, os contêineres de um sistema, suas redes, volumes e recursos, como o acesso à GPU. A operação de sistemas com componentes de aprendizado de máquina em produção exige ainda práticas de MLOps, como versionamento de configurações, monitoramento das saídas e reprodutibilidade das execuções (Kreuzberger; Kühl; Hirschl, 2023), que orientaram a forma como o sistema registra cada validação.

Por fim, a extração de texto de documentos PDF é feita pelo n8n com base na biblioteca PDF.js (Mozilla, 2026), que reconstrói o texto a partir dos objetos gráficos do documento. Como o PDF não armazena a estrutura lógica do texto, o resultado preserva as quebras de linha da diagramação, intercala cabeçalhos e rodapés de página com o conteúdo e não distingue as colunas de tabelas, características que influenciam o projeto da etapa de segmentação.

2.6 IA CENTRADA NO SER HUMANO

As seções anteriores trataram dos componentes técnicos do sistema; falta definir o papel das pessoas que usam seus resultados, já que, como visto na Seção 2.2, o modelo pode errar. Shneiderman (2022) defende que sistemas de IA devem ampliar a capacidade humana, combinando altos níveis de automação com altos níveis de controle humano, em vez de substituir o julgamento das pessoas. As diretrizes de interação humano-IA de Amershi *et al.* (2019) recomendam, entre outros pontos, deixar claro o que o sistema faz e com que qualidade, explicar por que ele tomou cada decisão e facilitar a correção quando ele erra. No campo educacional, o relatório do United States of America (2023) recomenda manter o ser humano no circuito decisório (*humans in the loop*) em qualquer uso de IA que afete estudantes e educadores.

Esses princípios orientaram três características do sistema: ele não aprova nem reprova planos de forma definitiva, apenas sinaliza possíveis não conformidades; cada aponta-

mento vem acompanhado de uma justificativa; e o painel oferece acesso direto ao PDF original para que o avaliador confirme ou descarte o apontamento.

2.7 MÉTRICAS DE AVALIAÇÃO

Manter o avaliador humano no circuito não dispensa medir a qualidade dos apontamentos do sistema, pois é dela que depende a confiança desse avaliador. Como o sistema atribui a cada critério uma classe, sua qualidade pode ser medida com as métricas usuais de classificação. Para avaliar um classificador, compara-se sua saída com uma referência considerada correta (*ground truth*). Neste trabalho, a tarefa é tratada como classificação binária por critério, em que a classe positiva é a não conformidade: o sistema acerta quando aponta um problema que de fato existe. As quatro situações possíveis compõem a matriz de confusão: verdadeiro positivo (VP), quando o sistema aponta uma não conformidade real; falso positivo (FP), quando aponta um problema inexistente; falso negativo (FN), quando deixa de apontar um problema real; e verdadeiro negativo (VN), quando considera conforme um item conforme (Sokolova; Lapalme, 2009). A partir delas são definidas as métricas das Equações 2.2 a 2.5 (Sokolova; Lapalme, 2009).

$$\text{Acurácia} = \frac{VP + VN}{VP + FP + FN + VN} \tag{2.2}$$

$$\text{Precisão} = \frac{VP}{VP + FP} \tag{2.3}$$

$$\text{Recall} = \frac{VP}{VP + FN} \tag{2.4}$$

$$F_1 = 2 \cdot \frac{\text{Precisão} \cdot \text{Recall}}{\text{Precisão} + \text{Recall}} \tag{2.5}$$

A precisão indica, entre os itens apontados como problemáticos, quantos realmente o eram; o *recall*, ou sensibilidade, indica quantos dos problemas existentes foram encontrados. Em uma ferramenta de triagem, o *recall* tende a ser mais importante, pois um falso negativo deixa passar um plano irregular, enquanto um falso positivo apenas gera uma revisão humana desnecessária. O F_1 é a média harmônica entre as duas.

Quando a classe positiva é rara, a acurácia pode ser alta mesmo para um classificador trivial. O coeficiente Kappa de Cohen corrige esse efeito ao descontar a concordância

esperada ao acaso (Cohen, 1960), conforme a Equação 2.6, em que p_o é a proporção de concordância observada e p_e é a proporção esperada ao acaso, calculada a partir das frequências marginais de cada classe.

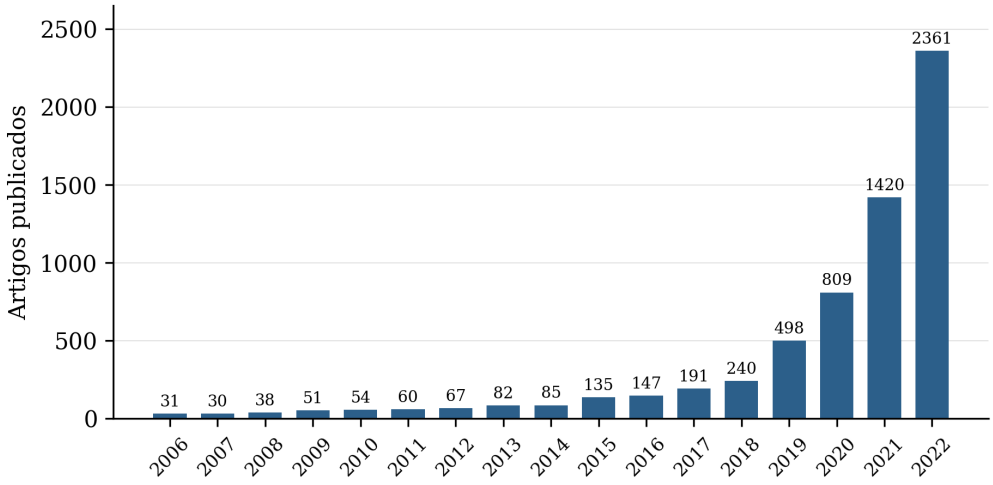
$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{2.6}$$

O Kappa vale 1 para concordância perfeita e 0 para concordância equivalente ao acaso. Landis e Koch (1977) propuseram a interpretação mais difundida para seus valores: abaixo de 0 indica concordância pobre; de 0,00 a 0,20, leve; de 0,21 a 0,40, razoável; de 0,41 a 0,60, moderada; de 0,61 a 0,80, substancial; e de 0,81 a 1,00, quase perfeita.

2.8 TRABALHOS RELACIONADOS

Definidos os conceitos técnicos e as métricas, esta seção situa o trabalho em relação à produção acadêmica sobre IA na educação, com atenção especial ao seu uso em processos administrativos. O emprego de IA na educação tem crescido de forma acentuada. A análise bibliométrica de Metli (2023), que examinou 6.498 artigos indexados na *Web of Science* entre 1984 e 2022, identificou crescimento anual de 22,68% nas publicações sobre IA e educação. Como mostra a Figura 1, a produção anual permaneceu abaixo de 100 artigos até 2014 e acelerou a partir de 2019, chegando a 2.361 artigos em 2022.

Figura 1 – Artigos publicados por ano sobre IA e educação (2006–2022)



Fonte: Elaborado pelo autor com dados de Metli (2023).

Dentro desse volume crescente de publicações, foram selecionados sete trabalhos que tratam do uso de IA no ensino e na gestão educacional, descritos a seguir. O primeiro é a

revisão sistemática de Tan, Cheng e Ling (2025), que analisou 95 publicações de 2015 a 2024 sobre IA no ensino e no desenvolvimento profissional docente e identificou aplicações em planejamento de aulas, avaliação automatizada, *feedback* individualizado e otimização de tarefas administrativas. Os autores apontam, porém, a escassez de estudos sobre processos administrativos de instituições de ensino superior, lacuna na qual este trabalho se insere.

O trabalho de Kasneci *et al.* (2023), um ensaio crítico, discute as oportunidades e os desafios dos LLMs na educação. Entre os riscos, os autores destacam a confiança excessiva nas respostas do modelo e a necessidade de supervisão humana e de transparência, preocupações que orientaram a decisão de manter o avaliador humano como responsável pela decisão final no sistema aqui desenvolvido.

Com foco na gestão, Sposato (2025) propõe uma taxonomia das aplicações de IA na liderança e na administração de instituições de ensino. A automação de análises documentais aparece nessa taxonomia entre os usos administrativos, mas de forma conceitual, sem a implementação de um sistema.

Já Hoare e Smith-Ferguson (2025) relatam a experiência do Departamento de Educação de Nova Gales do Sul, na Austrália, que desenvolveu uma ferramenta própria de IA generativa, o NSWEduChat, testada em 50 escolas a partir de 2024 para apoiar práticas de ensino baseadas em evidências. Entre os mais de 100 professores participantes da formação associada, 95% afirmaram que a ferramenta economizaria tempo. O estudo destaca que a contextualização das respostas com materiais e diretrizes institucionais foi decisiva para a confiança dos docentes, o que apoia a estratégia de fundamentar a avaliação nas normas do IFC.

No contexto brasileiro, Santos, Silva e Lima (2025) analisam a adoção de plataformas digitais e de IA na gestão de redes públicas de ensino e alertam que a centralização dessas plataformas pode restringir a autonomia dos atores escolares, concluindo que uma adoção responsável exige transparência no uso dos dados e infraestrutura adequada.

Mais próximo do contexto deste trabalho, Vieira e Moura (2025) realizaram uma revisão integrativa sobre IA nos Institutos Federais. Os autores constataram pouca produção acadêmica sobre o tema e identificaram a IA como apoio administrativo como a forma de incorporação menos explorada na rede federal.

Por fim, Vieira Filho (2024) aplica técnicas de PLN em Python à recomendação de conteúdo educacional em português. Embora não trate de processos administrativos, o trabalho demonstra a viabilidade dessas técnicas sobre textos acadêmicos no idioma, condição necessária

para a abordagem adotada aqui.

A Tabela 1 sintetiza a comparação entre os sete trabalhos descritos nesta seção e a proposta deste trabalho. A análise bibliométrica de Metli (2023) não foi incluída na comparação por tratar do volume de publicações sobre o tema, e não de uma aplicação específica de IA.

Tabela 1 – Comparação entre os trabalhos relacionados e este trabalho

Trabalho	**Foco**	**Tipo de estudo**	**Contexto**	**Processo administrativo?**
Tan, Cheng e Ling (2025)	IA no ensino e na formação docente	Revisão sistemática	Internacional	Apontado como lacuna
Kasneci *et al.* (2023)	Oportunidades e riscos de LLMs na educação	Ensaio crítico	Internacional	Não
Sposato (2025)	IA na gestão educacional	Taxonomia	Internacional	Sim, conceitual
Hoare e Smith-Ferguson (2025)	Ferramenta institucional de IA generativa	Relato de implantação	Austrália, educação básica	Não
Santos, Silva e Lima (2025)	Plataformas e IA na gestão pública	Análise documental	Brasil, redes públicas	Sim, conceitual
Vieira e Moura (2025)	IA nos Institutos Federais	Revisão integrativa	Brasil, rede federal	Apontado como lacuna
Vieira Filho (2024)	PLN para recomendação de conteúdo	Desenvolvimento de sistema	Brasil, ensino superior	Não
Este trabalho	**Validação de planos de ensino**	**Sistema e estudo de caso**	**IFC Videira**	**Sim, implementado e avaliado**

Fonte: Elaborado pelo autor (2026).

Os trabalhos revisados confirmam o interesse crescente pela IA na educação, mas tratam a gestão administrativa de forma conceitual ou a apontam como lacuna. Nenhum deles implementa e avalia um sistema para a verificação de planos de ensino frente a normas institucionais. Este trabalho busca contribuir para preencher essa lacuna com uma solução implementada, de baixo custo, executada localmente e avaliada sobre documentos reais de um Instituto Federal da região Sul.

3 METODOLOGIA

Este capítulo descreve como a pesquisa foi conduzida: sua classificação, as etapas realizadas, o corpus de documentos, os critérios de avaliação, o ambiente experimental, o protocolo de avaliação e os cuidados éticos adotados. O processo de desenvolvimento de software propriamente dito é detalhado na Seção 4.2.

3.1 CLASSIFICAÇÃO DA PESQUISA

Quanto à natureza, trata-se de uma pesquisa aplicada, pois busca gerar conhecimento para a solução de um problema concreto (Gil, 2017): a conferência dos planos de ensino de um curso de graduação. Quanto aos objetivos, é exploratória, por investigar a viabilidade de uma tecnologia ainda pouco aplicada a esse problema, e descritiva, por caracterizar o comportamento do sistema sobre documentos reais (Gil, 2017). A abordagem é predominantemente quantitativa, com métricas de desempenho e de concordância, complementada por uma análise qualitativa dos erros.

Quanto ao procedimento, adota-se o estudo de caso, que investiga um fenômeno contemporâneo em seu contexto real (Yin, 2015). O caso estudado é a validação dos planos de ensino do curso de Ciência da Computação do IFC – Campus Videira, e a unidade de análise é o par (plano de ensino, critério de avaliação).

3.2 ETAPAS DA PESQUISA

A partir dessa classificação, a pesquisa foi organizada em cinco etapas, relacionadas aos objetivos específicos:

1. Levantamento de critérios (primeiro objetivo específico): leitura da Organização Didática do IFC (Instituto Federal Catarinense, 2022) e dos planos exportados pelo SIGAA, e definição, em conjunto com o orientador, dos critérios de avaliação e de suas regras.

2. Projeto e implementação (segundo e terceiro objetivos específicos): desenvolvimento iterativo do fluxo de validação e do painel *web*, descrito no Capítulo 4.

3. Execuções sobre o corpus: processamento repetido dos planos de ensino a cada nova versão do sistema, com registro de todos os resultados.

4. Avaliação (quarto objetivo específico): cálculo das métricas de robustez, desempenho e concordância e análise dos erros.
5. Redação: consolidação dos resultados e elaboração deste documento.

3.3 CORPUS

As etapas de execução e de avaliação foram realizadas sobre um mesmo conjunto de documentos. O corpus é formado pelos 29 planos de ensino do semestre letivo 2024/1 do curso de Ciência da Computação disponibilizados em uma pasta do Google Drive para este trabalho. Todos foram exportados do SIGAA em PDF com texto pesquisável, o que dispensou o reconhecimento óptico de caracteres. Os documentos têm entre 3 e 5 páginas (média de 3,4) e, após a extração, entre 4.520 e 11.436 caracteres (média de 7.048, desvio-padrão de 1.700). Entre os componentes curriculares há disciplinas de formação básica, como matemática e estatística, de formação tecnológica, como programação, banco de dados e redes, e de formação complementar, como metodologia científica e trabalho de conclusão de curso. Dois planos correspondem a turmas distintas de um mesmo componente, o que permitiu observar a consistência do sistema diante de documentos quase idênticos.

3.4 CRITÉRIOS DE AVALIAÇÃO

Sobre cada plano do corpus, o sistema verifica um conjunto de critérios. Esses critérios foram definidos a partir dos elementos exigidos pela Organização Didática (Instituto Federal Catarinense, 2022) e das orientações do orientador deste trabalho, que sugeriu a lista inicial de itens a verificar e participou dos ajustes posteriores. Dois ajustes merecem registro. O item "adequação à ementa" foi incorporado ao critério de coerência, por se sobrepor a ele. A exigência de que os procedimentos de avaliação mencionassem a interdisciplinaridade foi removida, pois as primeiras execuções mostraram que ela levava o modelo a reprovar planos que descreviam adequadamente seus instrumentos de avaliação. Os doze critérios resultantes e suas regras estão na Tabela 2.

Tabela 2 – Critérios de avaliação e regras de conformidade

Critério	Regra para o status OK	Avaliado por
Identificação	Contém o nome da instituição, do componente curricular e do docente	LLM
Ementa	Descreve os temas da disciplina em texto corrido com ao menos 15 palavras	LLM
Objetivo geral	Possui texto preenchido	Regra
Objetivos específicos	Possuem texto preenchido	Regra
Conteúdo programático	Lista os tópicos ou unidades de conteúdo	LLM
Metodologia	Descreve como as aulas serão conduzidas (estratégias e recursos)	LLM
Procedimentos de avaliação	Descreve os instrumentos de avaliação	LLM
Cronograma	Apresenta distribuição temporal dos conteúdos (datas ou semanas)	LLM
Avaliações	Indica as avaliações previstas (instrumentos, datas ou pesos)	LLM
Horário de atendimento	Informa dia(s) e/ou horário(s) de atendimento ao estudante	LLM
Referências	No mínimo 3 referências básicas e 5 complementares	Regra
Coerência	Objetivos, metodologia e avaliação compatíveis entre si	LLM

Fonte: Elaborado pelo autor (2026), com base em Instituto Federal Catarinense (2022).

Cada critério recebe um entre três status: OK, quando atende à regra; INCOMPLETO, quando há conteúdo relacionado que não atende à regra; e AUSENTE, quando o campo está vazio ou sem relação com o critério. O plano recebe um status geral: OK, se todos os critérios estão OK; PENDENTE, se há ao menos um critério INCOMPLETO e nenhum AUSENTE; e REPROVADO, se há ao menos um critério AUSENTE. Falhas técnicas que impeçam a avaliação geram o status ERRO.

3.5 AMBIENTE EXPERIMENTAL

Para que os resultados de desempenho possam ser interpretados e reproduzidos, é preciso registrar o equipamento em que foram obtidos. Todas as execuções foram realizadas em um computador portátil, com os componentes descritos na Tabela 3. O modelo de linguagem foi executado inteiramente no equipamento local, sem envio dos documentos a serviços de IA externos.

Tabela 3 – Ambiente experimental

Componente	Especificação
Processador	Intel Core Ultra 7 155H
Memória RAM	32 GB
GPU	NVIDIA GeForce RTX 4050 Laptop, 6 GB de VRAM
Sistema operacional	Windows 11, com Docker Desktop
Orquestração	n8n 2.21.7 (contêiner `docker.n8n.io/n8nio/n8n`)
Servidor de inferência	Ollama 0.24.0 (contêiner `ollama/ollama`)
Modelo de linguagem	Qwen2.5 7B Instruct, 7,6 bilhões de parâmetros, quantização Q4_K_M
Parâmetros de geração	temperatura 0,1; contexto de 8.192 *tokens*; máximo de 1.500 *tokens* gerados
Otimizações do Ollama	*FlashAttention* ativado; *KV cache* quantizado em 8 bits (`q8_0`)
Armazenamento	Google Drive (planos) e Google Sheets (resultados) nas execuções avaliadas; pastas locais na versão final (Seção 4.1)

Fonte: Elaborado pelo autor (2026).

A temperatura de 0,1 foi escolhida para tornar as respostas praticamente determinísticas, característica desejável em uma tarefa de verificação, na qual o mesmo documento deve receber a mesma avaliação em execuções diferentes.

3.6 PROTOCOLO DE AVALIAÇÃO

Definidos o corpus, os critérios e o ambiente, resta descrever como o sistema foi avaliado, o que corresponde ao quarto objetivo específico. A avaliação foi organizada em quatro dimensões, cada uma com uma fonte de dados e uma medida próprias, descritas a seguir.

Robustez técnica. Neste trabalho, robustez técnica é a capacidade do sistema de processar todos os planos do lote sem falhas que impeçam a avaliação, como respostas vazias, JSON inválido ou estouro do tempo limite. Ela foi medida pela proporção de planos que receberam o status ERRO em cada execução sobre o corpus, acompanhada do motivo da falha registrado pelo sistema. Como a planilha de resultados usada durante o desenvolvimento acumulou todas as execuções, foi possível reconstruir a evolução dessa proporção ao longo das iterações de desenvolvimento e associar cada queda a uma correção específica.

Desempenho computacional. Para cada requisição ao modelo, o Ollama informa o número de *tokens* de entrada e de saída e o tempo gasto em cada fase. Esses metadados, preservados no histórico de execuções do n8n, foram usados para calcular o tempo de inferência por plano, a velocidade de geração em *tokens* por segundo e o tempo total de processamento do lote.

Concordância com oráculo estrutural. Para o critério de horário de atendimento, a

presença de conteúdo pode ser verificada de forma objetiva no texto extraído: no modelo do SIGAA, o rótulo “Horário de Atendimento” é seguido imediatamente pela seção “Programa”, e o texto entre os dois indica se o campo foi preenchido. Esse oráculo estrutural, implementado no *script* do Apêndice C, serve como referência independente do modelo de linguagem para o cálculo das métricas da Seção 2.7. Para os critérios de referências e de objetivos, avaliados por regras determinísticas, verificou-se por amostragem, no texto extraído, se as contagens e detecções correspondiam ao conteúdo dos documentos.

Análise de erros. Todos os planos com algum critério diferente de OK na execução final foram confrontados com o texto extraído do PDF, classificando cada apontamento como correto ou incorreto e identificando a causa dos erros.

Para os critérios que dependem de interpretação, como metodologia e coerência, não existe oráculo estrutural, e a referência adequada é o parecer de avaliadores humanos. O protocolo previsto para essa comparação consiste em: (i) solicitar a membros do NDE que classifiquem, de forma independente e sem acesso à saída do sistema, cada critério de cada plano; (ii) consolidar os pareceres em uma referência única, resolvendo divergências por consenso; e (iii) calcular, por critério, precisão, *recall*, F_1 e Kappa de Cohen entre o sistema e a referência. Os artefatos necessários para executar esse protocolo, a planilha de anotação e o *script* de cálculo das métricas, foram produzidos e estão descritos no Apêndice C. A realização da anotação pelo NDE não coube no período deste trabalho e é discutida como limitação no Capítulo 6.

3.7 ASPECTOS ÉTICOS

Por trabalhar com documentos reais, a pesquisa exigiu cuidados com a exposição das pessoas envolvidas. Os planos de ensino são documentos institucionais de caráter público para a comunidade acadêmica, mas contêm o nome dos docentes responsáveis. Para evitar qualquer exposição individual, os resultados deste trabalho são apresentados de forma anonimizada: os planos são identificados por códigos de P01 a P29, atribuídos pela ordem alfabética do nome do arquivo, e nenhuma tabela associa resultados a docentes ou a componentes curriculares. O processamento foi realizado em equipamento local, sem envio dos documentos a provedores externos de IA, em consonância com os princípios de finalidade e de necessidade da LGPD (Brasil, 2018). Os resultados do sistema têm caráter indicativo e não foram utilizados para qualquer decisão sobre os planos avaliados.

4 DESENVOLVIMENTO

Este capítulo documenta o sistema desenvolvido. A Seção 4.1 apresenta uma visão geral da solução e a Seção 4.2 descreve o processo de desenvolvimento adotado. As Seções 4.3 a 4.7 especificam os requisitos, os atores, as regras de negócio e as histórias de usuário. A Seção 4.8 apresenta a modelagem de dados, a Seção 4.9 reúne os diagramas estruturais e comportamentais, a Seção 4.10 detalha a implementação de cada etapa e a Seção 4.11 apresenta a interface. Por fim, a Seção 4.12 descreve a estratégia de testes.

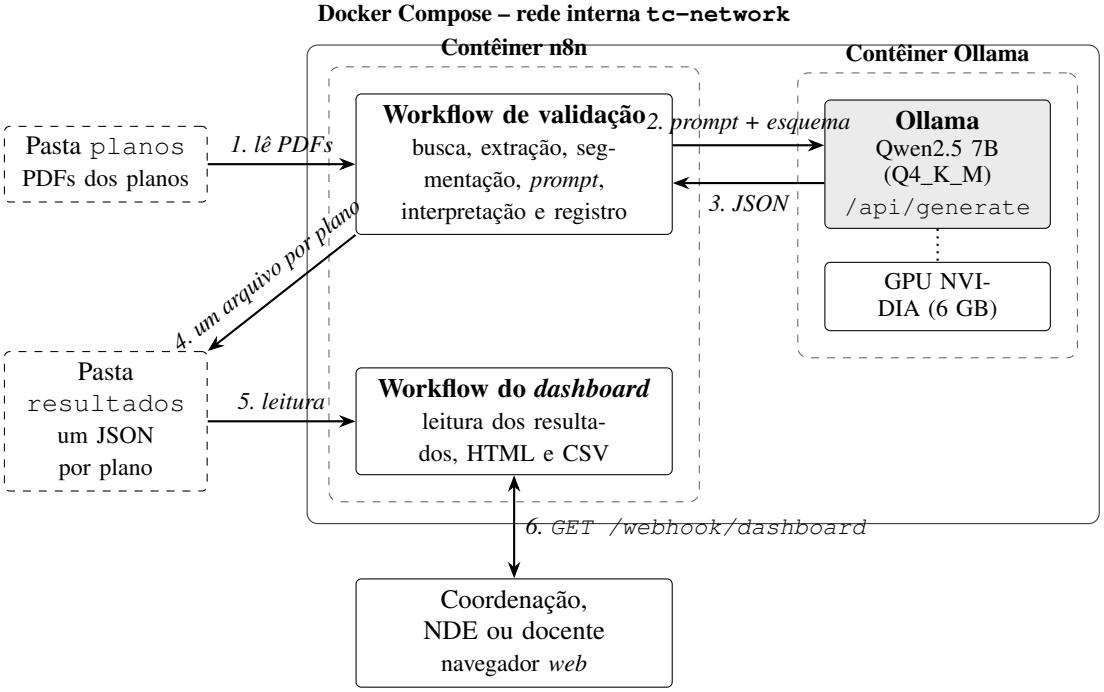
4.1 APRESENTAÇÃO DO SISTEMA

O sistema desenvolvido é um conjunto de dois fluxos de automação executados na plataforma n8n, apoiados por um servidor de modelos de linguagem local. O fluxo de validação lê os planos de ensino em PDF de uma pasta local, extrai e segmenta o texto de cada plano, solicita ao modelo de linguagem a avaliação dos critérios, aplica regras determinísticas sobre a resposta e grava o resultado de cada plano em um arquivo JSON, em outra pasta local. O fluxo do *dashboard* lê esses arquivos e gera, a cada acesso, uma página HTML com os indicadores do lote, o detalhamento de cada plano e a exportação dos resultados em CSV.

A Figura 2 apresenta a arquitetura da solução. Os dois serviços próprios, n8n e Ollama, são executados em contêineres Docker orquestrados por um arquivo Docker Compose e se comunicam por uma rede interna. O Ollama tem acesso à GPU do computador hospedeiro, onde o modelo Qwen2.5 é carregado. Duas pastas do computador hospedeiro são montadas no contêiner do n8n: a pasta `planos`, repositório de entrada onde a coordenação deposita os PDFs, e a pasta `resultados`, onde o sistema grava as validações. Dessa forma, o sistema não depende de serviços externos nem de credenciais, e a implantação se resume a iniciar os contêineres, importar os fluxos e copiar os planos para a pasta de entrada. Até a penúltima iteração, a entrada e a saída usavam o Google Drive e o Google Sheets, acessados por OAuth 2.0. Essa configuração exigia, a cada implantação, criar um projeto no Google Cloud, ativar as APIs, configurar a tela de consentimento e cadastrar as credenciais no n8n, etapa apontada na revisão do orientador como complexa demais para a adoção do sistema, e por isso foi substituída pelo armazenamento local.

Três princípios orientaram o projeto. O primeiro é a divisão de trabalho entre regras e modelo: o que pode ser verificado de forma exata, como contar referências, é feito por código,

Figura 2 – Arquitetura do sistema



Fonte: Elaborado pelo autor (2026).

e o LLM é reservado aos julgamentos que exigem interpretação. O segundo é a verificabilidade: cada critério recebe um status de um conjunto fechado e uma observação curta, e o *dashboard* oferece acesso direto ao PDF original. O terceiro é o controle humano: o sistema faz uma triagem e não toma decisões, cabendo à coordenação e ao NDE confirmar ou descartar cada apontamento.

4.2 METODOLOGIA DE DESENVOLVIMENTO DE SOFTWARE

A solução apresentada na seção anterior não foi definida de uma só vez, mas construída ao longo de várias versões. O desenvolvimento seguiu um processo iterativo e incremental, no qual o sistema é construído em versões sucessivas e cada versão é avaliada antes de orientar a seguinte (Sommerville, 2016; Pressman; Maxim, 2016). A escolha se justifica pelo caráter exploratório do projeto: não era possível prever, antes de executar o sistema sobre planos reais, como o modelo de linguagem se comportaria diante dos documentos, quais regras seriam ambíguas ou quais limites de infraestrutura seriam atingidos. Em linha com os valores do Manifesto Ágil (Beck *et al.*, 2001), privilegiou-se ter software funcionando desde a primeira iteração e responder às mudanças identificadas nas execuções, em vez de seguir um plano fechado.

Cada iteração seguiu o mesmo ciclo: (i) executar a versão corrente sobre o corpus completo; (ii) inspecionar, nos resultados gravados e no histórico de execuções do n8n, os re-

sultados e as falhas; (iii) identificar a causa raiz do problema mais grave; e (iv) implementar e documentar a correção. O código dos fluxos, exportado do n8n em formato JSON, e a configuração da infraestrutura foram versionados com Git em um repositório no GitHub, e um arquivo de documentação registrou as instruções de instalação e os problemas conhecidos com suas soluções. A Tabela 4 resume as iterações realizadas.

Tabela 4 – Iterações de desenvolvimento

Período	**Principais entregas e correções**	**Execução de referência**
Maio/2026	Primeira versão do fluxo: busca no Drive, extração de texto, avaliação do plano pelo LLM e registro na planilha	22/05: 86,2% de ERRO
Julho/2026	Critérios definidos com o orientador; segmentação do texto por seções; *dashboard* HTML; suporte a GPU; documentação de instalação	07/07: 3,4% de ERRO
Agosto/2026	Processamento em blocos com uma requisição por vez e retentativas; adoção do Qwen2.5 7B; correção do contexto máximo do modelo (de 4.096 para 12.288 *tokens*)	10/08: 69,0%; 25/08: 49,1% de ERRO
Setembro/2026	Contagem determinística de referências; regras determinísticas para objetivos e status geral; remoção da exigência de interdisciplinaridade	10/09: 0%; 15/09: 17,2% de ERRO
Outubro/2026	Segmentação ancorada no início da linha; contexto de 8.192 *tokens*; *FlashAttention* e *KV cache* em 8 bits; *timeout* de 180 s	19/10: 0% de ERRO
Novembro/2026	Armazenamento local: planos lidos de uma pasta e resultados gravados em arquivos JSON, em substituição ao Google Drive e ao Google Sheets; exportação em CSV e abertura do PDF pelo *dashboard*	Teste funcional do fluxo, sem nova execução sobre o corpus

Fonte: Elaborado pelo autor (2026), com base no histórico de execuções e na planilha de resultados.

Os problemas encontrados em cada iteração e o efeito das correções são analisados na Seção 5.1.

4.3 REQUISITOS FUNCIONAIS

Os requisitos funcionais descrevem os serviços que o sistema deve oferecer (Sommerville, 2016). Eles foram levantados a partir do processo manual de conferência e refinados ao longo das iterações. A Tabela 5 apresenta os requisitos funcionais implementados.

Tabela 5 – Requisitos funcionais

Nº	Descrição do Requisito Funcional
RF01	O sistema deve ler os arquivos PDF de uma pasta local configurada, ignorando arquivos de outros tipos.
RF02	O sistema deve extrair o texto de cada plano de ensino em PDF.
RF03	O sistema deve segmentar o texto nas seções do plano: identificação, ementa, metodologia, procedimentos de avaliação, horário de atendimento, objetivo geral, objetivos específicos, conteúdo programático, cronograma, avaliações e referências.
RF04	O sistema deve contar as referências básicas e complementares de cada plano.
RF05	O sistema deve avaliar os doze critérios da Tabela 2, atribuindo a cada um o status OK, INCOMPLETO ou AUSENTE e uma observação.
RF06	O sistema deve calcular o status geral do plano (OK, PENDENTE ou REPROVADO) a partir dos status dos critérios.
RF07	O sistema deve registrar o status ERRO e o motivo da falha quando não for possível avaliar um plano.
RF08	O sistema deve gravar o resultado de cada plano em um arquivo na pasta de resultados, com data e hora da validação e o *link* para o PDF original.
RF09	O sistema deve executar a validação automaticamente uma vez por dia e permitir a execução manual.
RF10	O sistema deve disponibilizar um *dashboard* com o total de planos e as quantidades e percentuais de planos aprovados, pendentes e com problemas.
RF11	O *dashboard* deve exibir, para cada plano, o status e a observação de cada critério.
RF12	O *dashboard* deve permitir abrir o PDF original de cada plano.
RF13	O sistema deve permitir trocar o modelo de linguagem utilizado, inclusive por um serviço externo.
RF14	O *dashboard* deve permitir exportar todos os resultados em formato CSV.

Fonte: Elaborado pelo autor (2026).

4.4 REQUISITOS NÃO FUNCIONAIS

Enquanto os requisitos funcionais definem o que o sistema faz, os requisitos não funcionais expressam restrições e qualidades do sistema (Sommerville, 2016). Eles foram organizados segundo as características de qualidade da norma ISO/IEC 25010 (International Organization for Standardization, 2011) e estão na Tabela 6.

Tabela 6 – Requisitos não funcionais

Nº	Característica	Descrição do Requisito Não Funcional
RNF01	Segurança	O conteúdo dos planos não deve ser enviado a serviços externos de IA na configuração padrão; o modelo deve ser executado localmente.
RNF02	Segurança	Os fluxos devem ter acesso apenas às pastas de planos e de resultados, e a configuração padrão não deve depender de credenciais de serviços externos.
RNF03	Eficiência de desempenho	O sistema deve validar um lote de 30 planos em menos de 30 minutos no ambiente da Tabela 3.
RNF04	Eficiência de desempenho	O sistema deve operar com uma GPU de 6 GB de VRAM.
RNF05	Confiabilidade	A falha na avaliação de um plano não deve interromper o processamento dos demais.
RNF06	Confiabilidade	Requisições ao modelo devem ser repetidas até 3 vezes em caso de falha, com limite de tempo por requisição.
RNF07	Confiabilidade	Os resultados de cada bloco de planos devem ser gravados antes do processamento do bloco seguinte.
RNF08	Adequação funcional	A resposta do modelo deve seguir um esquema JSON com campos e valores predefinidos.
RNF09	Usabilidade	O *dashboard* deve funcionar em navegadores atuais, sem instalação, e indicar o status por cor e por texto.
RNF10	Portabilidade	O sistema deve ser implantável com Docker Compose em Windows, Linux ou macOS; a GPU deve ser opcional.
RNF11	Manutenibilidade	As regras de avaliação e o modelo devem ser configuráveis em um único ponto do fluxo.
RNF12	Manutenibilidade	Os fluxos e a infraestrutura devem ser versionados em repositório Git.

Fonte: Elaborado pelo autor (2026).

4.5 ATORES DO SISTEMA

Os requisitos descritos nas seções anteriores atendem a diferentes perfis de usuário. A Tabela 7 apresenta esses usuários e o que cada um pode fazer no sistema. Além deles, o agendador do n8n atua como ator temporal, disparando a validação diariamente.

Tabela 7 – Atores do sistema

Nº	Nome	Descrição
US1	Coordenação de curso / NDE	Deposita os planos na pasta de entrada, dispara ou aguarda a validação, consulta o *dashboard*, analisa os apontamentos e decide sobre cada plano.
US2	Docente	Consulta o resultado do próprio plano no *dashboard*, identifica os itens apontados e reenvia o plano corrigido.
US3	Administrador técnico	Instala e mantém a infraestrutura, configura as pastas de planos e de resultados e o modelo de linguagem, e acompanha as execuções no n8n.

Fonte: Elaborado pelo autor (2026).

4.6 REGRAS DE NEGÓCIO

Além do que cada ator pode fazer, é preciso definir como o sistema deve decidir. As regras de negócio orientam e controlam esse comportamento. Elas derivam da Organização Didática (Instituto Federal Catarinense, 2022), das orientações da coordenação e das decisões tomadas durante as iterações:

- RN01 – Cada critério recebe exatamente um status entre OK, INCOMPLETO e AUSENTE; havendo conteúdo relacionado ao critério, o status AUSENTE não deve ser usado.
- RN02 – O plano é REPROVADO se algum critério estiver AUSENTE; PENDENTE se algum estiver INCOMPLETO e nenhum AUSENTE; e OK se todos estiverem OK.
- RN03 – O critério de referências é conforme somente com no mínimo 3 referências básicas e 5 complementares, contadas pelo sistema e não pelo modelo.
- RN04 – Os critérios de objetivo geral e de objetivos específicos são conformes quando possuem texto preenchido, independentemente da redação.
- RN05 – Os procedimentos de avaliação não são considerados incompletos pela ausência de menção à interdisciplinaridade.
- RN06 – A avaliação de cada critério deve se basear apenas no texto do plano, sem inferência de informações ausentes.
- RN07 – O status calculado pelo sistema é indicativo; a decisão sobre o plano cabe à coordenação e ao NDE.
- RN08 – Uma falha técnica na avaliação de um plano gera o status ERRO, com o motivo registrado, e o plano deve ser reavaliado em nova execução.

4.7 HISTÓRIAS DE USUÁRIO

Os requisitos, os atores e as regras de negócio foram reunidos, do ponto de vista dos usuários, em histórias de usuário. As histórias de usuário descrevem funcionalidades do ponto de vista de quem as utiliza, no formato “como <papel>, quero <funcionalidade>, para <benefício>”, acompanhadas de critérios de aceitação (Cohn, 2004). A Tabela 8 relaciona as histórias que representam o escopo implementado.

Tabela 8 – Histórias de usuário

Nº	História	Critérios de aceitação
HU01	Como coordenador, quero validar todos os planos de uma pasta de uma só vez, para não precisar abrir documento por documento.	Todos os PDFs da pasta geram um registro de resultado; arquivos de outros tipos são ignorados. (RF01, RF02, RF08)
HU02	Como coordenador, quero ver quantos planos estão aprovados, pendentes ou com problemas, para dimensionar o trabalho de revisão.	O *dashboard* exibe total, quantidades e percentuais por status. (RF10)
HU03	Como membro do NDE, quero ver o status e a justificativa de cada critério de um plano, para revisar apenas os itens apontados.	Ao expandir um plano, os doze critérios aparecem com status, cor e observação. (RF05, RF11)
HU04	Como membro do NDE, quero abrir o PDF original a partir do resultado, para confirmar ou descartar um apontamento.	Cada plano possui um *link* que abre o arquivo original no navegador. (RF12)
HU05	Como coordenador, quero que a contagem de referências seja exata, para cobrar o mínimo exigido com segurança.	A observação informa a quantidade de referências básicas e complementares encontradas. (RF04, RN03)
HU06	Como coordenador, quero que a validação rode sozinha todos os dias, para que planos novos sejam avaliados sem intervenção.	O fluxo é disparado diariamente às 5h. (RF09)
HU07	Como docente, quero saber exatamente o que falta no meu plano, para corrigi-lo antes do prazo.	Cada critério não conforme tem observação que indica o problema. (RF05, RF11)
HU08	Como administrador, quero que uma falha em um plano não interrompa os demais e fique registrada, para reprocessar apenas o necessário.	O plano com falha recebe status ERRO com o motivo; os demais seguem. (RF07, RNF05)
HU09	Como administrador, quero trocar o modelo de linguagem em um único lugar, para testar modelos maiores ou um serviço externo.	O modelo é definido por uma constante no nó de preparação do *prompt*. (RF13, RNF11)
HU10	Como coordenador, quero levar os resultados para uma planilha eletrônica, para filtrá-los e compartilhá-los.	O *dashboard* oferece o *download* de um arquivo CSV com todos os resultados. (RF14)

Fonte: Elaborado pelo autor (2026).

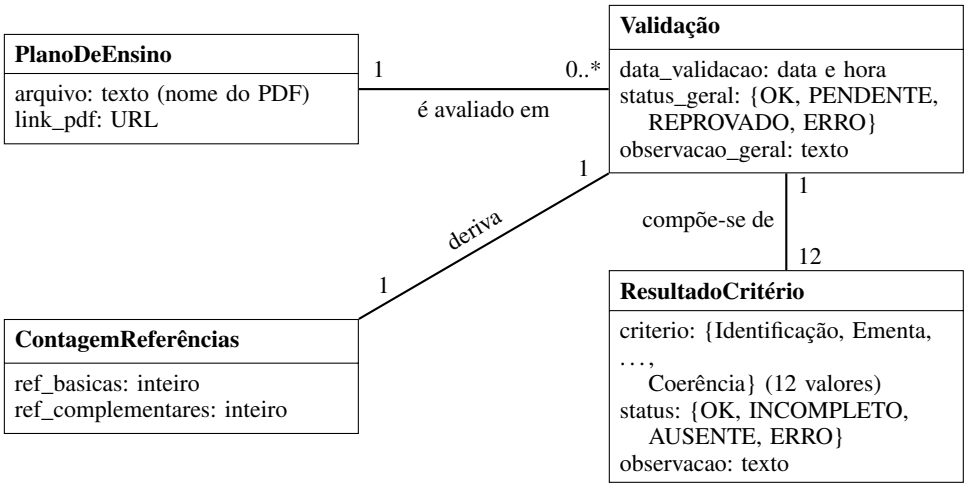
4.8 MODELAGEM DO BANCO DE DADOS

As funcionalidades descritas nas histórias de usuário dependem de dados persistidos, como os resultados de cada validação. O sistema não utiliza um sistema gerenciador de banco de dados relacional: os resultados são armazenados em arquivos JSON na pasta `resultados`, um arquivo por plano, nomeado a partir do nome do PDF. A escolha elimina a infraestrutura de banco de dados e as credenciais de serviços externos, e a exportação em CSV oferecida pelo *dashboard* permite que a coordenação consulte, filtre e compartilhe os resultados em uma planilha eletrônica já conhecida. Os planos de ensino, por sua vez, permanecem na pasta `planos`.

A Figura 3 apresenta o modelo conceitual dos dados. Um plano de ensino pode ser avaliado em várias execuções, e cada validação compõe-se de exatamente doze resultados de

critério e de uma contagem de referências.

Figura 3 – Modelo conceitual dos dados



Fonte: Elaborado pelo autor (2026).

Na implementação, esse modelo é desnormalizado em um único registro por plano, em que os doze resultados de critério são representados por pares de campos (status e observação), totalizando 29 campos, descritos na Tabela 9. A desnormalização simplifica a gravação, feita com a escrita de um único arquivo por plano, e a leitura pelo *dashboard*. Como o arquivo de um plano é sobrescrito a cada nova validação, a pasta reflete sempre o resultado mais recente de cada plano, sem registros duplicados. Durante o desenvolvimento, quando os resultados eram gravados no Google Sheets, cada execução acrescentava novas linhas sem apagar as anteriores, e foi esse histórico acumulado que permitiu reconstruir a evolução do sistema apresentada no Capítulo 5.

Tabela 9 – Estrutura do registro de resultado de cada plano

Campo	Tipo	Conteúdo
Arquivo	Texto	Nome do arquivo PDF na pasta de planos
Link PDF	URL	Endereço do *webhook* que abre o PDF original
Data Validação	Data e hora	Momento da validação (fuso de Brasília)
Status Geral	Enumeração	OK, PENDENTE, REPROVADO ou ERRO
Observação Geral	Texto	Resumo em uma frase ou motivo da falha
<Critério> (12 campos)	Enumeração	OK, INCOMPLETO, AUSENTE ou ERRO
<Critério> Obs (12 campos)	Texto	Justificativa do status do critério

Fonte: Elaborado pelo autor (2026).

4.9 DIAGRAMAS

Definidos os requisitos e os dados, esta seção apresenta os diagramas que descrevem a estrutura e o comportamento do sistema, elaborados com base na notação da UML (Object Management Group, 2017). Como a solução é composta por fluxos de automação e não por um programa orientado a objetos, foram selecionados os diagramas que melhor representam seus elementos: o diagrama de fluxo do *pipeline*, como visão estrutural complementar à arquitetura da Figura 2, e os diagramas de casos de uso, de sequência, de atividades e de estados, como visões comportamentais.

4.9.1 Diagramas Estruturais

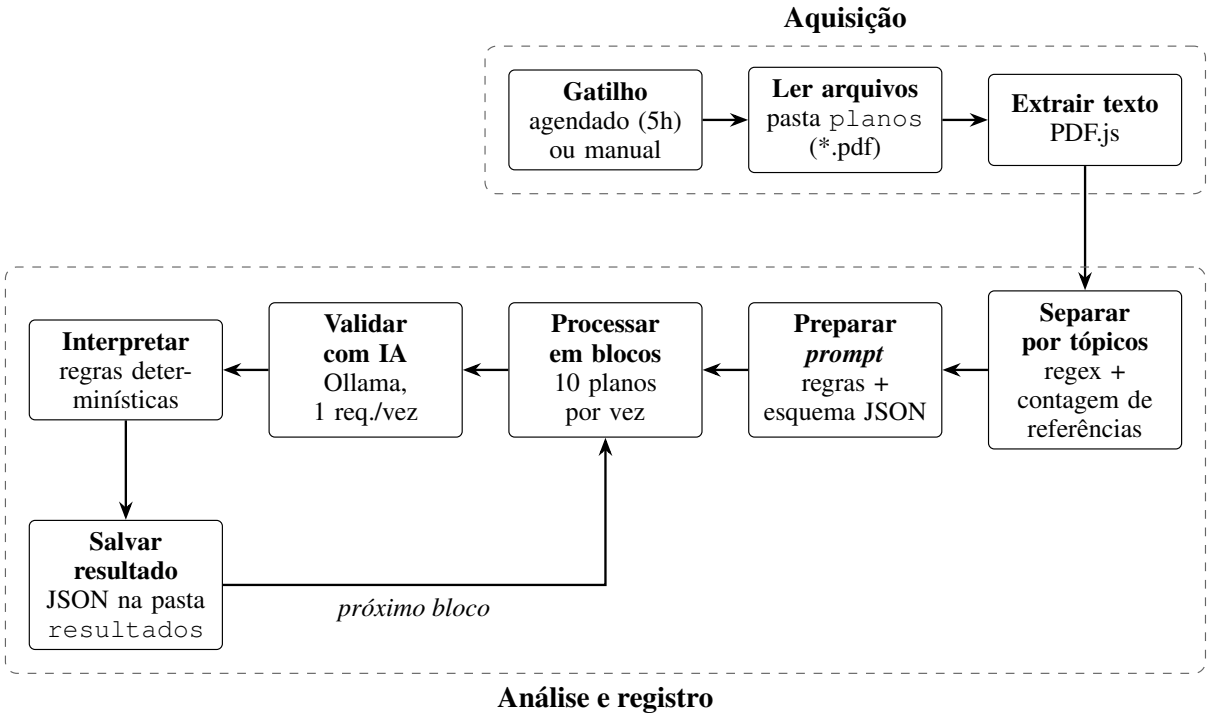
A Figura 4 apresenta a estrutura do fluxo de validação no n8n. Cada caixa corresponde a um nó do fluxo. A etapa de aquisição é disparada por um gatilho agendado ou manual, lê os arquivos PDF da pasta de planos e extrai o texto. A etapa de análise e registro separa o texto em seções, prepara o *prompt* e então entra em um laço controlado pelo nó *Processar em blocos*: cada bloco de até dez planos é enviado ao modelo, um plano por requisição; as respostas são interpretadas e gravadas na pasta de resultados; e só então o bloco seguinte é processado. Esse laço garante que uma interrupção no meio do lote não descarte os resultados já obtidos (RNF07).

O fluxo do *dashboard* é mais simples e tem dois caminhos. No primeiro, um *webhook* recebe a requisição HTTP, um nó lê os arquivos da pasta de resultados, outro converte o conteúdo JSON, um nó de código gera a página HTML e um último nó devolve a página ao navegador. No segundo, um *webhook* recebe o nome de um plano, lê o PDF correspondente da pasta de planos e o devolve ao navegador, o que permite abrir o documento original a partir do *dashboard*.

4.9.2 Diagramas Comportamentais

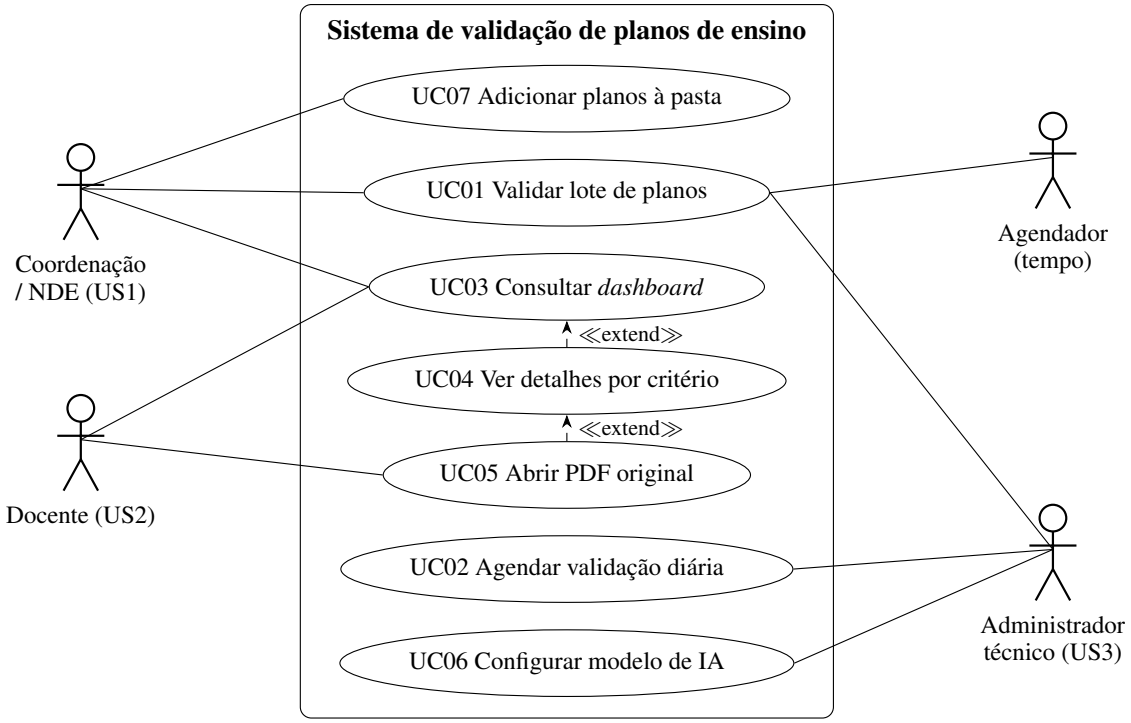
Enquanto os diagramas estruturais mostram como o sistema está organizado, os comportamentais mostram como ele é usado e como reage a cada situação. A Figura 5 apresenta o diagrama de casos de uso. A coordenação e o NDE validam lotes, adicionam planos à pasta e consultam o *dashboard*; o docente consulta o *dashboard* e abre o PDF original; o administrador configura o agendamento e o modelo; e o agendador dispara a validação diária. Ver o detalhe por critério e abrir o PDF são extensões da consulta ao *dashboard*.

Figura 4 – Estrutura do fluxo de validação no n8n



Fonte: Elaborado pelo autor (2026).

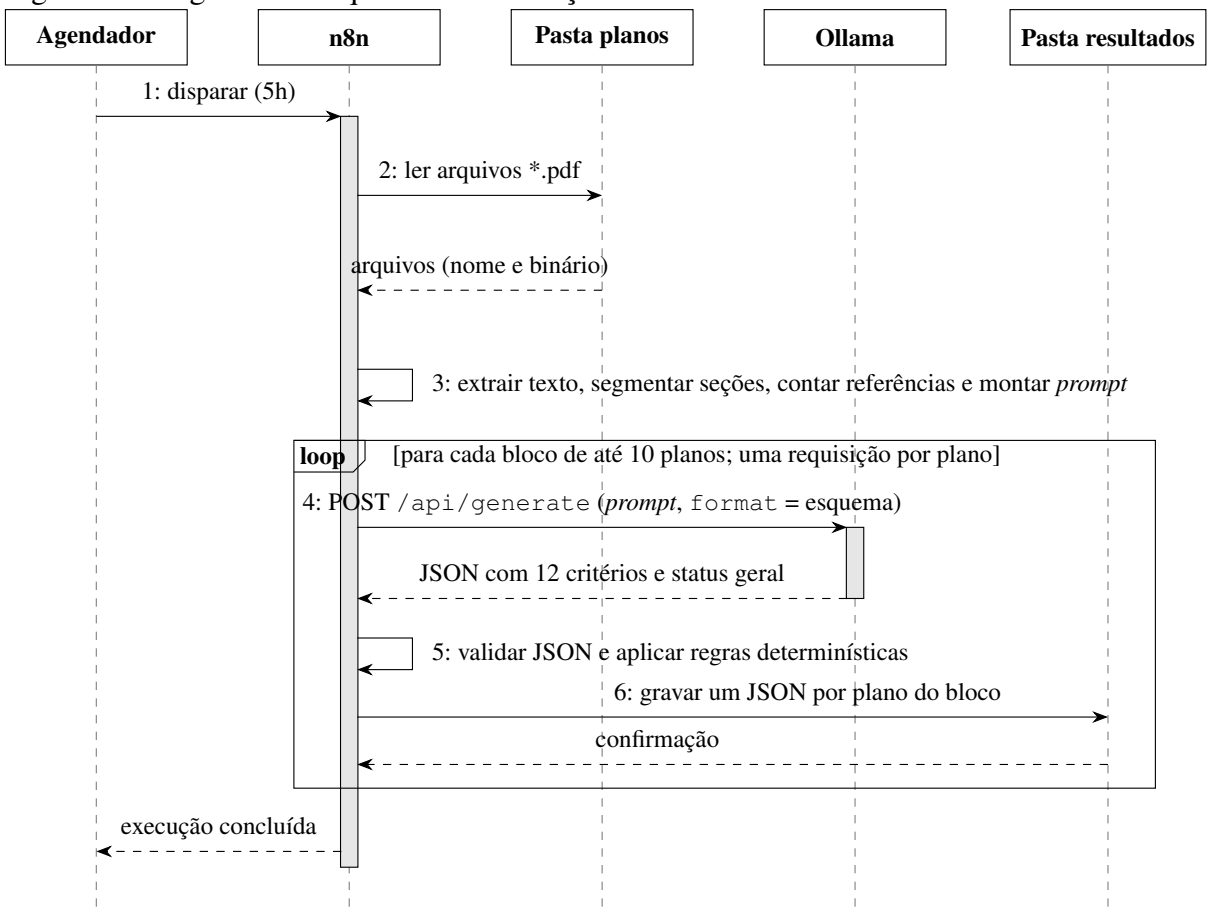
Figura 5 – Diagrama de casos de uso



Fonte: Elaborado pelo autor (2026).

A Figura 6 apresenta o diagrama de sequência de uma execução de validação. Após o disparo, o n8n lê os PDFs da pasta de planos e executa a extração, a segmentação e a montagem dos *prompts*. No laço, envia cada plano ao Ollama com o esquema JSON no parâmetro `format`, interpreta a resposta e, ao fim de cada bloco, grava um arquivo de resultado por plano na pasta de resultados.

Figura 6 – Diagrama de sequência da validação

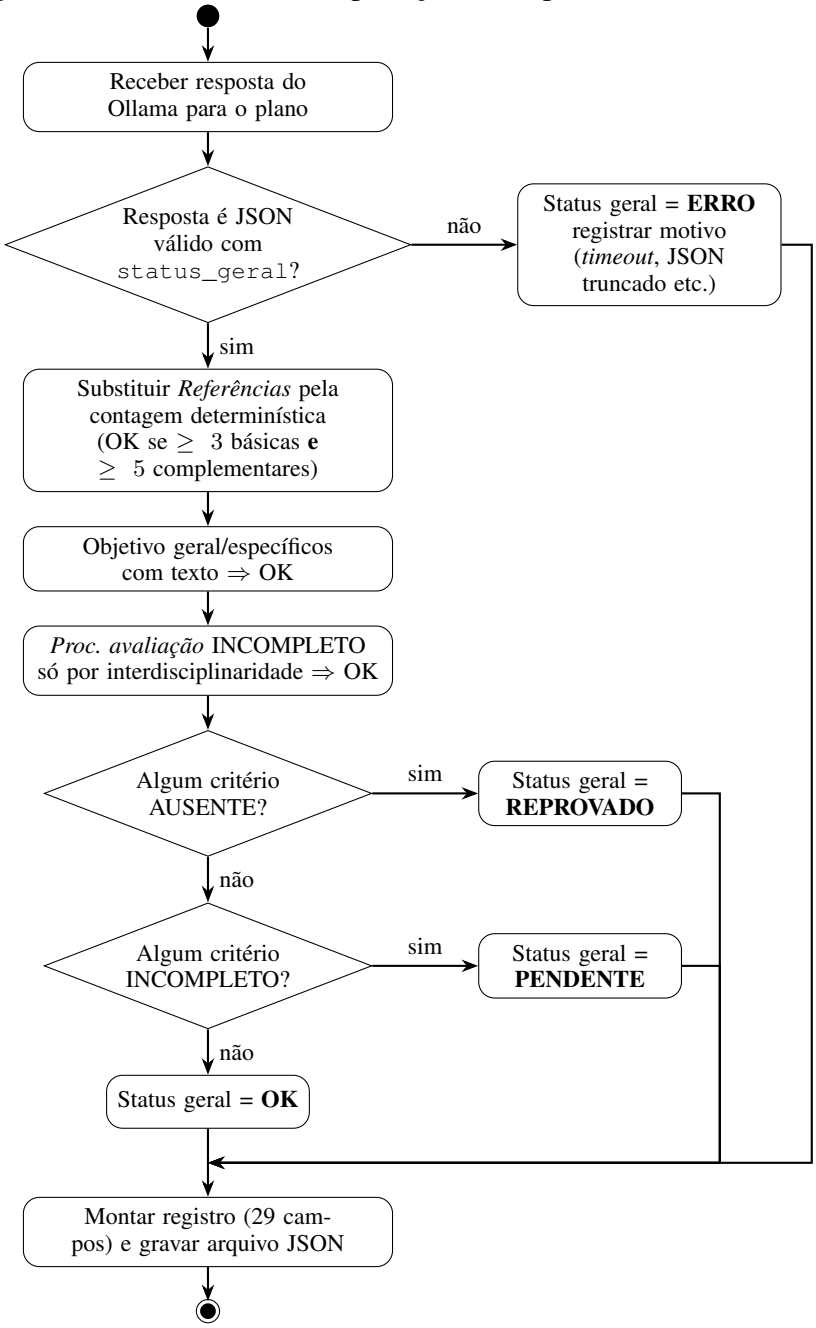


Fonte: Elaborado pelo autor (2026).

A Figura 7 detalha, em um diagrama de atividades, a interpretação da resposta do modelo para um plano. Se a resposta não for um JSON válido, ou se a requisição tiver falhado, o plano recebe o status ERRO com o motivo. Caso contrário, as regras determinísticas RN03 a RN05 substituem o julgamento do modelo nos critérios correspondentes e o status geral é recalculado por código, segundo a RN02, em vez de se aceitar o valor sugerido pelo modelo.

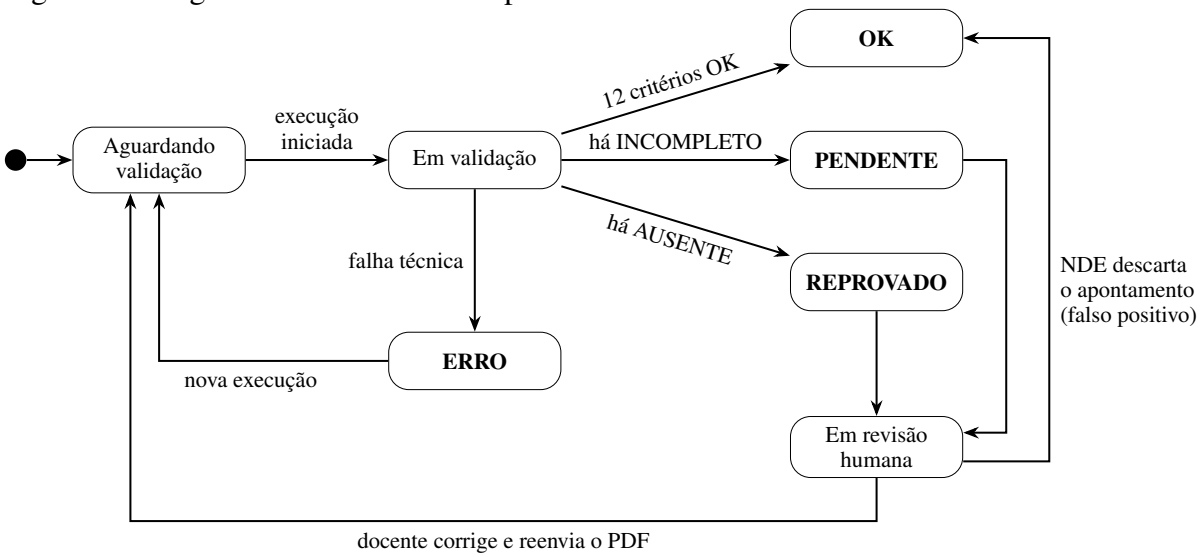
A Figura 8 apresenta o ciclo de vida de um plano de ensino no processo apoiado pelo sistema. Planos PENDENTES ou REPROVADOS seguem para revisão humana, que pode levar à correção e ao reenvio do plano pelo docente ou ao descarte do apontamento pelo NDE, quando se tratar de um falso positivo. Planos com ERRO voltam a aguardar validação.

Figura 7 – Diagrama de atividades da interpretação da resposta



Fonte: Elaborado pelo autor (2026).

Figura 8 – Diagrama de estados de um plano de ensino



Fonte: Elaborado pelo autor (2026).

4.10 IMPLEMENTAÇÃO

Os diagramas da seção anterior descrevem o sistema em nível de projeto. Esta seção descreve como cada etapa do fluxo de validação foi implementada e justifica as principais decisões técnicas. Os trechos de código mais relevantes estão nos Apêndices A e B.

4.10.1 Infraestrutura

Todas as etapas do fluxo dependem da infraestrutura que executa o n8n e o modelo de linguagem, por isso ela é descrita primeiro. A infraestrutura é definida em um arquivo Docker Compose com dois serviços na mesma rede: o n8n, publicado na porta 5678, e o Ollama, publicado na porta 11434 e com acesso à GPU por meio do recurso de reserva de dispositivos do Docker. Volumes persistentes preservam o histórico do n8n e os modelos baixados pelo Ollama, e as pastas `planos` e `resultados` do hospedeiro são montadas no contêiner do n8n, que tem o acesso a arquivos restrito a elas pela variável `N8N_RESTRICT_FILE_ACCESS_TO`. No Ollama foram configurados o *FlashAttention* (`OLLAMA_FLASH_ATTENTION=1`), a quantização do *KV cache* em 8 bits (`OLLAMA_KV_CACHE_TYPE=q8_0`) e a permanência do modelo na memória por 30 minutos após o último uso, evitando recarregá-lo entre requisições. No n8n, o tempo máximo de execução de nós de código foi ampliado para 30 minutos, pois um bloco com retentativas pode ultrapassar o limite padrão.

4.10.2 Aquisição e extração de texto

Sobre essa infraestrutura, o fluxo de validação começa pela obtenção dos documentos. O fluxo usa o nó *Read/Write Files from Disk* para ler, de uma só vez, todos os arquivos com extensão `.pdf` da pasta de planos, o que dispensa um filtro por tipo de arquivo. O texto é extraído pelo nó *Extract from File*, que utiliza o PDF.js (Mozilla, 2026). O texto resultante preserva as quebras de linha da diagramação e intercala, a cada página, o cabeçalho e o rodapé institucionais com o conteúdo, como se observa no trecho do Apêndice B.

4.10.3 Segmentação e contagem de referências

Como o texto extraído mistura o conteúdo das seções com cabeçalhos e rodapés, a etapa seguinte é separá-lo nas seções do plano. A segmentação é feita por um nó de código JavaScript que localiza no texto os rótulos das seções do modelo do SIGAA por meio de expressões regulares e recorta o conteúdo de cada seção desde o fim de seu rótulo até o início do rótulo seguinte. Duas decisões foram decisivas para a qualidade desse recorte. Primeiro, os padrões são ancorados no início da linha e tolerantes a variações de caixa e de acentuação, pois, sem a âncora, uma frase como "As avaliações serão…" no meio dos procedimentos de avaliação era tomada como o início da seção *Avaliações*, truncando a primeira e preenchendo a segunda com texto errado. Segundo, cada seção termina no início do cabeçalho seguinte, e não no início de seu conteúdo, para que nenhum rótulo vaze para a seção anterior. A seção de objetivos é ainda dividida em objetivo geral e objetivos específicos a partir de marcadores como "Objetivo geral" e "Específicos".

A contagem de referências também é feita por expressão regular. No PDF do SIGAA, as referências aparecem em uma tabela com as colunas "Tipo de Material" e "Descrição", de modo que cada referência começa, no texto extraído, por um tipo de material como "Livro", "Periódico" ou "Site". O sistema conta essas ocorrências nos blocos de referências básicas e complementares. A decisão de retirar a contagem do modelo decorreu da observação, nas primeiras iterações, de que o LLM errava contagens de linhas de tabela com frequência, o que é consistente com a dificuldade desses modelos em operações exatas discutida na Seção 2.2.

4.10.4 Construção do *prompt* e saída estruturada

Com as seções recortadas e as referências contadas, o sistema reúne essas informações na requisição enviada ao modelo. Para cada plano é montado um único *prompt*, repro-

duzido no Apêndice A, com quatro partes: (i) a persona de avaliador de planos de ensino e a instrução de avaliar apenas com base no texto fornecido, sem inventar informações; (ii) a definição dos três status e a restrição de nunca usar AUSENTE quando houver conteúdo relacionado; (iii) a regra de cada critério, correspondente à Tabela 2; e (iv) o texto de cada seção, truncado em 1.500 caracteres, ou 2.500 para conteúdo programático e cronograma, acompanhado da contagem de referências já verificada. Avaliar o plano seção por seção, em vez de entregar o documento inteiro, reduz o ruído dos cabeçalhos e rodapés e coloca cada trecho junto do rótulo a que pertence, mitigando a perda de informação em contextos longos (Liu *et al.*, 2024).

O mesmo nó gera o esquema JSON enviado no parâmetro `format` da requisição ao Ollama. O esquema exige um objeto com os doze critérios, cada um com os campos `status`, restrito à enumeração OK, INCOMPLETO e AUSENTE, e `obs`, além de `status_geral` e `obs_geral`. Como discutido na Seção 2.3, o esquema faz com que a validade estrutural da resposta seja garantida pela decodificação (Willard; Louf, 2023; Ollama, 2024). O nó também gera um *payload* equivalente para a API do Google Gemini, o que permite trocar o modelo local por um serviço externo alterando apenas a URL e o corpo da requisição (RF13).

4.10.5 Chamada ao modelo e interpretação da resposta

Montados o *prompt* e o esquema, resta enviá-los ao modelo e tratar a resposta. A requisição ao Ollama é feita por um nó HTTP com limite de 180 segundos, até três tentativas com intervalo de 5 segundos e envio de uma requisição por vez. O envio sequencial foi adotado porque requisições simultâneas faziam o servidor de inferência enfileirar e interromper respostas, produzindo JSON truncado.

A interpretação, cujo comportamento foi apresentado na Figura 7, extrai o objeto JSON da resposta, verifica a presença do status geral e aplica as regras determinísticas. Quando a requisição falha, o motivo informado pelo nó HTTP, como o estouro do tempo limite, é registrado na observação geral, o que facilitou o diagnóstico dos problemas durante as iterações. Por fim, o nó monta o registro de 29 campos da Tabela 9, gravado como arquivo JSON na pasta de resultados pelo nó de escrita de arquivos.

4.10.6 *Dashboard*

Os arquivos gravados pelo fluxo de validação são a fonte de dados do segundo fluxo, o do *dashboard*. A página é gerada por um nó de código que recebe todos os registros da

pasta de resultados e produz uma página HTML autocontida, sem dependências externas. Todo conteúdo proveniente dos registros é escapado antes de ser inserido na página, evitando que textos dos planos ou das observações sejam interpretados como HTML. O uso de elementos HTML nativos de expansão (`details` e `summary`) permite exibir o detalhe de cada plano sem JavaScript. A página traz ainda um atalho para baixar todos os resultados em um arquivo CSV, gerado no próprio nó de código, que abre em planilhas eletrônicas e serve à análise dos resultados fora do sistema.

4.11 PROTOTIPAÇÃO

O *dashboard*, cuja implementação foi descrita acima, é a interface do sistema com o usuário. Ele é acessado pelo caminho de *webhook* `dashboard` do n8n e apresentado na Figura 9 com os dados da execução final. No topo, quatro indicadores mostram o total de planos e as quantidades e percentuais de aprovados, pendentes e com problemas, estes últimos somando os planos reprovados e com erro. Abaixo, cada plano é um cartão com o nome, o status geral e o atalho para o PDF. Ao clicar em um cartão, são exibidos a observação geral e os doze critérios, cada um com uma faixa e um selo coloridos e a observação correspondente. As cores seguem a convenção verde para OK, amarelo para INCOMPLETO e PENDENTE, e vermelho para AUSENTE, REPROVADO e ERRO, sempre acompanhadas do texto do status, para não depender apenas da cor (RNF09).

O plano P05, expandido na figura, ilustra o uso pretendido: o avaliador vê imediatamente que faltam o horário de atendimento e os objetivos específicos e que o conteúdo programático está incompleto, e pode abrir o PDF para confirmar. O mesmo exemplo revela também uma limitação discutida na Seção 5.5: o objetivo geral aparece como OK, embora o campo contenha apenas o texto provisório “Não Cadastrado!” do SIGAA.

4.12 TESTES

Para garantir que cada etapa descrita neste capítulo funcionasse como esperado ao longo das iterações, a estratégia de testes combinou três abordagens. Os testes de execução, feitos a cada alteração de um nó, usaram o modo de execução manual do n8n, que exibe a entrada e a saída de cada nó e permite inspecionar o resultado da extração, da segmentação, do *prompt* e da resposta do modelo para um plano específico. Os testes de regressão, feitos ao fim de cada iteração, consistiram em processar novamente o corpus completo e comparar os resul-

Figura 9 – *Dashboard* de validação com o plano P05 expandido



Fonte: Elaborado pelo autor (2026). Nomes dos arquivos substituídos por códigos para anonimização.

tados com os da execução anterior, verificando se a correção resolvia o problema visado sem introduzir novos erros. Por fim, a avaliação da versão final, segundo o protocolo da Seção 3.6, mediu robustez, desempenho e concordância. Os resultados dessas avaliações são apresentados no Capítulo 5.

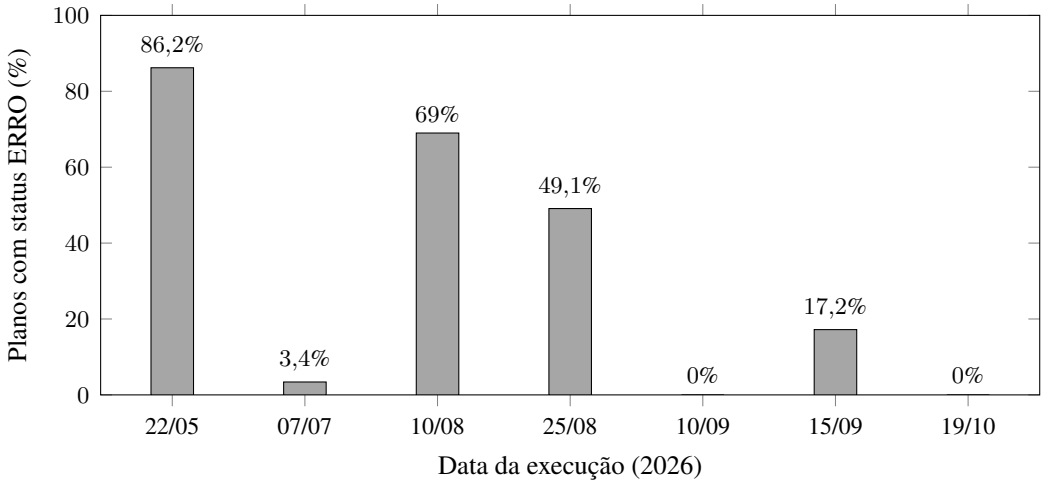
5 AVALIAÇÃO E RESULTADOS

Este capítulo apresenta e discute os resultados obtidos segundo o protocolo da Seção 3.6. A Seção 5.1 analisa a robustez técnica ao longo das iterações, a Seção 5.2 trata do desempenho computacional, a Seção 5.3 apresenta os resultados da validação dos 29 planos, a Seção 5.4 mede a concordância com o oráculo estrutural e a Seção 5.5 analisa os erros do sistema. A Seção 5.6 discute os resultados em conjunto. Salvo indicação contrária, os números referem-se à execução da versão final, realizada em 19 de outubro de 2026.

5.1 ROBUSTEZ TÉCNICA AO LONGO DAS ITERAÇÕES

A primeira dimensão avaliada é a robustez técnica, pois um sistema que falha ao processar os planos não pode ter seus resultados avaliados nas demais dimensões. A Figura 10 mostra a proporção de planos que receberam o status ERRO em cada data de execução registrada na planilha de resultados. Nas datas com mais de uma execução, como 25/08, o valor agrega todas as execuções do dia.

Figura 10 – Proporção de planos com status ERRO por data de execução



Fonte: Elaborado pelo autor (2026), a partir da planilha de resultados (348 registros) e da execução de 19/10/2026.

A análise das mensagens de erro registradas permitiu identificar quatro causas de falha, resumidas na Tabela 10 com as correções correspondentes.

Tabela 10 – Causas de falha identificadas e correções aplicadas

Sintoma	Causa	Correção
Resposta vazia ou JSON truncado (“*Unexpected end of JSON input*”)	Requisições simultâneas sobrecarregavam o servidor de inferência, que interrompia respostas	Processamento em blocos, uma requisição por vez, com até três tentativas
JSON malformado (“*Expected ’,’ or ’}’*”)	O modelo produzia texto fora da sintaxe JSON	Saída estruturada com esquema JSON no parâmetro `format`
Respostas em outro idioma ou com JSON sem relação com o pedido	O *prompt* (cerca de 5.100 *tokens* à época) excedia o contexto padrão de 4.096 *tokens* e era truncado	Definição explícita do contexto (12.288 e, depois, 8.192 *tokens*) e redução do *prompt*
Estouro do tempo limite (*timeout*)	Geração lenta, com parte do modelo fora da GPU e queda de desempenho do equipamento	Contexto de 8.192, *FlashAttention* e *KV cache* em 8 bits

Fonte: Elaborado pelo autor (2026).

A causa mais frequente foi a resposta vazia ou truncada, responsável por todas as falhas de 22/05 e de 10/08 e por 55 das 57 falhas de 25/08. O problema não estava no modelo, mas na forma de chamá-lo: o n8n enviava as requisições de todos os planos quase ao mesmo tempo, e o servidor as enfileirava e interrompia. A solução foi controlar a concorrência no próprio fluxo, com o laço de blocos da Figura 4. O comportamento intermitente, com poucas falhas em 07/07 e muitas novamente em 10/08, é compatível com uma falha dependente da carga sobre o servidor, e não de planos específicos. As duas falhas de 07/07 foram de outro tipo, JSON malformado, eliminado com a adoção da saída estruturada.

O truncamento do contexto foi o problema mais difícil de diagnosticar, pois não gerava erro: o Ollama descartava silenciosamente a parte do *prompt* que excedia o contexto padrão, e o modelo, sem parte das instruções, respondia em outro idioma ou produzia um JSON sem relação com o pedido. O episódio ilustra uma característica dos sistemas com LLMs: falhas de configuração podem se manifestar como respostas plausíveis e incorretas, e não como erros explícitos, o que reforça a importância do registro detalhado das execuções defendido pelas práticas de MLOps (Kreuzberger; Kühl; Hirschl, 2023).

Na execução de 15/09, cinco planos falharam por estouro do tempo limite, então de 600 segundos. Os metadados do Ollama mostram que a velocidade de geração caiu de 18,1 para 5,6 *tokens* por segundo em relação à execução de 10/09, com a mesma configuração de inferência, o que indica uma queda de desempenho do equipamento, como o estrangulamento térmico ou a entrada em modo de economia de energia, e não um problema do fluxo. A otimização de memória da iteração de outubro, discutida na próxima seção, e o tempo limite menor, de 180

segundos, com retentativas, tornaram o sistema menos sensível a essas variações. Na execução final, nenhum dos 29 planos falhou.

5.2 DESEMPENHO COMPUTACIONAL

Eliminadas as falhas técnicas, a questão seguinte é se o sistema processa o lote em tempo compatível com o uso real. A Tabela 11 resume os metadados de inferência da execução final. Cada plano gerou, em média, um *prompt* de 3.248 *tokens* e uma resposta de 274 *tokens*. A leitura do *prompt* levou em média 1,6 segundo e a geração da resposta, 9,9 segundos, totalizando 11,5 segundos de inferência por plano. A velocidade média de geração foi de 27,6 *tokens* por segundo. O lote completo de 29 planos, incluindo a listagem e o *download* dos arquivos, a extração, as chamadas ao modelo e a gravação na planilha, foi processado em 8 minutos e 54 segundos, abaixo do limite de 30 minutos estabelecido no RNF03. Essa execução usou a versão com Google Drive e Google Sheets; na versão final, com pastas locais, a leitura dos arquivos e a gravação dos resultados deixam de passar pela rede, e o tempo total continua dominado pela inferência, que respondeu por cerca de 5,6 dos 8,9 minutos.

Tabela 11 – Estatísticas de inferência por plano na execução final (n = 29)

Medida	**Mínimo**	**Máximo**	**Média**	**Mediana**	**Desvio-padrão**
Páginas do PDF	3	5	3,4	3	0,6
Caracteres extraídos	4.520	11.436	7.048	6.770	1.700
Tokens do *prompt*	2.481	4.522	3.248	3.120	558
Tokens da resposta	250	347	274	269	18
Tempo de leitura do *prompt* (s)	0,8	11,3	1,6	1,2	1,9
Tempo de geração (s)	8,9	12,3	9,9	9,8	0,6

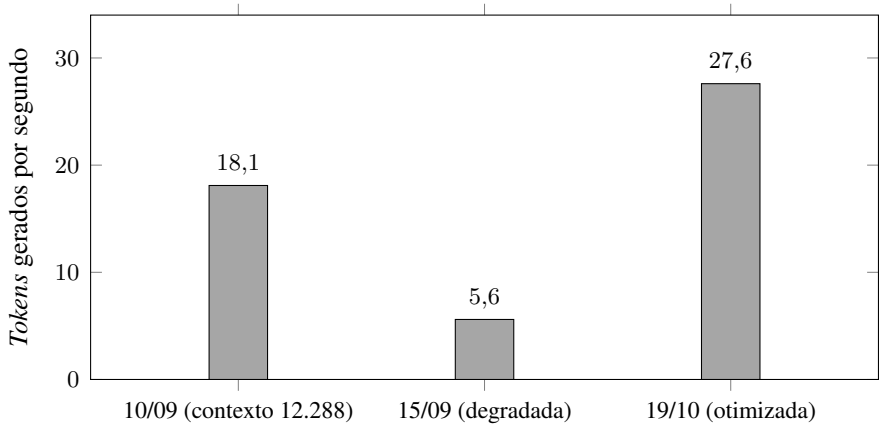
Fonte: Elaborado pelo autor (2026), a partir dos metadados do Ollama.

O tamanho da resposta variou pouco, pois o esquema JSON fixa sua estrutura, e o tempo de geração acompanhou essa estabilidade, com desvio-padrão de apenas 0,6 segundo. O tempo de leitura do *prompt* teve um valor atípico de 11,3 segundos em um dos planos; nos demais, ficou entre 0,8 e 1,9 segundo.

A Figura 11 compara a velocidade de geração em três execuções sobre o mesmo corpus. Entre 10/09 e 19/10, a velocidade passou de 18,1 para 27,6 *tokens* por segundo, um ganho de 52%, e o tempo total do lote caiu de 46 minutos para menos de 9 minutos. O ganho decorre da combinação de três mudanças na iteração de outubro: a redução do contexto de 12.288 para 8.192 *tokens*, viabilizada pela segmentação mais precisa, que eliminou trechos

desnecessários do *prompt*; o *FlashAttention*; e a quantização do *KV cache* em 8 bits. Juntas, essas mudanças reduziram a memória ocupada pelo contexto; na configuração final, o Ollama reportou 92% do modelo alocado na GPU de 6 GB, com o restante na CPU. A execução de 15/09 é mostrada como referência do efeito da degradação do equipamento discutida na seção anterior.

Figura 11 – Velocidade de geração por configuração



Fonte: Elaborado pelo autor (2026), a partir dos metadados do Ollama.

Extrapolando a média de 11,5 segundos de inferência por plano, a validação dos planos de todos os cursos de um campus, na casa das centenas por semestre, levaria menos de uma hora no mesmo equipamento, executada de madrugada pelo agendamento diário. O desempenho obtido é, portanto, compatível com o uso real, sem custo por requisição e sem envio dos documentos a serviços externos.

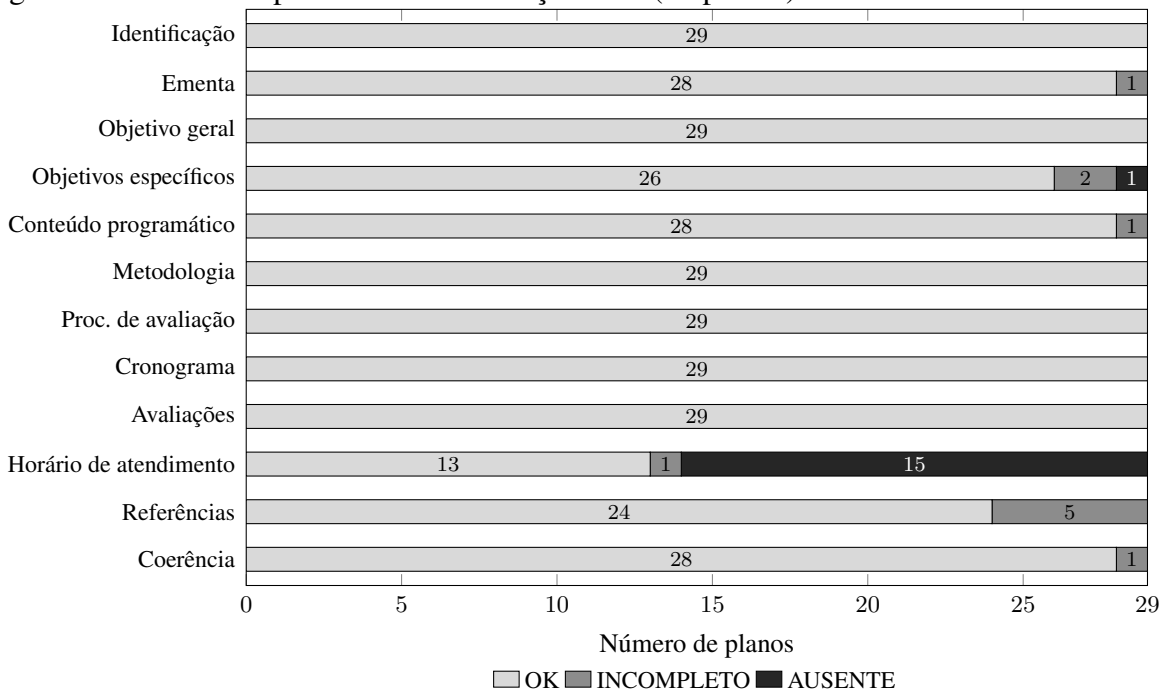
5.3 RESULTADOS DA VALIDAÇÃO DO CORPUS

Com o sistema estável e com desempenho adequado, esta seção apresenta o que ele apontou sobre os planos. Na execução final, 11 planos (37,9%) foram considerados conformes (OK), 3 (10,3%) pendentes e 15 (51,7%) reprovados. A Figura 12 detalha o resultado por critério.

Seis critérios foram considerados conformes em todos os planos: identificação, objetivo geral, metodologia, procedimentos de avaliação, cronograma e avaliações. O resultado é esperado para identificação e cronograma, cuja estrutura é gerada pelo próprio SIGAA, e indica que os docentes descrevem de forma consistente a metodologia e os instrumentos de avaliação.

O critério com mais não conformidades foi o horário de atendimento, ausente em 15 planos e incompleto em um. Todos os 15 planos reprovados têm o horário de atendimento

Figura 12 – Resultado por critério na execução final (29 planos)



Fonte: Elaborado pelo autor (2026).

ausente, e em 9 deles essa é a única não conformidade. O dado é relevante para a gestão do curso: trata-se de uma informação de interesse direto dos estudantes, de preenchimento simples, cuja ausência em mais da metade dos planos dificilmente seria percebida sem uma verificação sistemática.

O segundo critério mais apontado foi o de referências, incompleto em 5 planos. Em quatro deles faltavam referências complementares: três planos tinham 4 e um não tinha nenhuma das 5 exigidas. No quinto, a proporção estava invertida, com 5 referências básicas e apenas 3 complementares. Os demais apontamentos foram pontuais: objetivos específicos em três planos, e ementa, conteúdo programático e coerência em um plano cada.

5.4 CONCORDÂNCIA COM O ORÁCULO ESTRUTURAL

Os resultados da seção anterior só são úteis se os apontamentos estiverem corretos. O critério de horário de atendimento, o mais apontado, é também o único que pode ser conferido por uma referência independente do modelo. Para esse critério, o oráculo estrutural descrito na Seção 3.6 identificou 15 planos com o campo vazio e 14 com conteúdo. A Tabela 12 apresenta a matriz de confusão do sistema em relação a esse oráculo, considerando como positivo o plano não conforme, isto é, com status INCOMPLETO ou AUSENTE.

Tabela 12 – Matriz de confusão do critério horário de atendimento

	Sistema: não conforme	**Sistema: conforme**	**Total**
Oráculo: campo vazio	15 (VP)	0 (FN)	15
Oráculo: campo preenchido	1 (FP)	13 (VN)	14
Total	16	13	29

Fonte: Elaborado pelo autor (2026).

O sistema detectou todos os 15 campos vazios, o que corresponde a um *recall* de 100%, e apontou um único plano cujo campo estava preenchido, resultando em precisão de 93,8%, F_1 de 0,968 e acurácia de 96,6%. O Kappa de Cohen foi de 0,931, o que corresponde a concordância quase perfeita na escala de Landis e Koch (1977).

O único falso positivo em relação ao oráculo, o plano P19, merece atenção. Seu campo contém a frase "conforme horário disponibilizado publicamente", que não informa dia nem horário. Pela regra do critério (Tabela 2), o plano é de fato não conforme, de modo que o apontamento está correto quanto à conformidade, e a divergência decorre da limitação do oráculo, que apenas verifica se o campo está vazio. O erro do modelo está na categoria: deveria ter atribuído INCOMPLETO, pois há conteúdo relacionado, mas atribuiu AUSENTE, contrariando a instrução explícita do *prompt*. Considerando a regra do critério, e não apenas o oráculo estrutural, o sistema acertou a conformidade dos 29 planos e errou a categoria em dois: o P19, como descrito, e o P21, que tem o campo vazio e recebeu INCOMPLETO em vez de AUSENTE.

5.5 ANÁLISE DE ERROS

Para os demais critérios, que não têm oráculo estrutural, a correção dos apontamentos foi verificada manualmente. Todos os apontamentos da execução final, além dos de horário de atendimento, foram confrontados com o texto extraído dos PDFs. A Tabela 13 apresenta o resultado dessa verificação.

Tabela 13 – Verificação dos apontamentos da execução final (exceto horário de atendimento)

Plano	Critério	Sistema	Conteúdo no PDF	Avaliação
P05	Objetivos específicos	AUSENTE	“Objetivos: Não Cadastrado!”	Correto
P05	Conteúdo programático	INCOMPLETO	“Não Cadastrado!”	Não conformidade correta; categoria deveria ser AUSENTE
P05	Coerência	INCOMPLETO	Sem objetivos para comparar	Correto
P05	Objetivo geral	OK	“Não Cadastrado!”	Falso negativo
P20, P21	Objetivos específicos	INCOMPLETO	Objetivo em parágrafo único, sem objetivos específicos	Não conformidade correta
P27	Ementa	INCOMPLETO	“A cadastrar”	Não conformidade correta; categoria deveria ser AUSENTE
P11, P23, P25, P28, P29	Referências	INCOMPLETO	Contagens conferidas no texto	Correto

Fonte: Elaborado pelo autor (2026).

Todos os 11 apontamentos verificados correspondem a não conformidades reais. Em dois deles, porém, a categoria atribuída foi INCOMPLETO quando o campo continha apenas um texto provisório do SIGAA, situação que deveria ser classificada como AUSENTE. Esse padrão, somado aos dois erros de categoria do horário de atendimento, mostra que o modelo distingue bem a presença de um problema, mas é menos confiável na escolha entre INCOMPLETO e AUSENTE. O caso dos planos P20 e P21, duas turmas do mesmo componente com o campo de horário igualmente vazio, que receberam AUSENTE e INCOMPLETO respectivamente, mostra que essa escolha pode variar mesmo com temperatura 0,1, quando outras partes do *prompt*, como o cronograma, diferem. Como a categoria determina se o plano é REPROVADO ou PENDENTE, essa instabilidade afeta o status geral: o P21 foi classificado como PENDENTE, enquanto o P20, praticamente idêntico, foi REPROVADO.

Foi identificado um falso negativo: o objetivo geral do plano P05 foi considerado OK, embora contivesse apenas o texto provisório “Não Cadastrado!”. O erro não é do modelo, mas da regra determinística RN04, que considera conforme qualquer texto preenchido. Uma busca por textos provisórios em todo o corpus confirmou que esse foi o único caso em que um texto provisório foi aceito como conteúdo válido; nos outros dois (conteúdo programático do P05 e ementa do P27), o modelo apontou corretamente a não conformidade.

Por fim, a análise revelou um problema de consistência na observação geral, o resumo em uma frase produzido pelo modelo. Em 18 dos 24 planos cujo critério de referências

foi considerado conforme, a observação geral mencionava algum problema com as referências, como “faltam referências básicas”. A contradição ocorre porque a observação geral é escrita pelo modelo antes da aplicação das regras determinísticas, e o modelo, mesmo recebendo a contagem verificada, tende a comentar as referências de forma equivocada. Embora não altere nenhum status, a contradição aparece no *dashboard* e pode confundir o avaliador. A correção indicada é gerar a observação geral por código, a partir dos status finais dos critérios.

5.6 DISCUSSÃO

As seções anteriores analisaram cada dimensão da avaliação separadamente. Considerados em conjunto, os resultados permitem responder ao problema de pesquisa sob três aspectos. Quanto à viabilidade técnica, o sistema processou o corpus completo sem falhas, em menos de 9 minutos, em um computador portátil com GPU de 6 GB e sem custos por requisição. A evolução das iterações mostra, porém, que essa robustez não veio do modelo, mas da engenharia em torno dele: controle de concorrência, saída estruturada, dimensionamento do contexto e registro detalhado das execuções. Esse resultado é coerente com a visão de Kreuzberger, Kühl e Hirschl (2023) de que sistemas com componentes de aprendizado de máquina dependem tanto da infraestrutura e da operação quanto do modelo.

Quanto à qualidade da triagem, o sistema não deixou de apontar nenhuma das não conformidades verificáveis por oráculo e todos os seus apontamentos verificados correspondiam a problemas reais, com um único falso negativo identificado, causado por uma regra determinística e não pelo modelo. A principal fragilidade está na distinção entre INCOMPLETO e AUSENTE. Para uma ferramenta de triagem, cujo objetivo é dirigir a atenção do avaliador, a detecção do problema é mais importante que sua categoria, mas a categoria afeta o status geral e, portanto, a priorização dos planos. Esses resultados estão alinhados com a literatura sobre LLMs como avaliadores, que relata boa concordância com humanos acompanhada de vieses e inconsistências que precisam ser controlados (Zheng *et al.*, 2023; Chang *et al.*, 2024).

Quanto à divisão de trabalho entre regras e modelo, os resultados confirmam a decisão de projeto. Os critérios mais objetivos, como a contagem de referências, tornaram-se exatos quando passaram a ser verificados por código, enquanto, nos critérios interpretativos, todos os apontamentos feitos pelo modelo foram confirmados na verificação, inclusive o de coerência. Ao mesmo tempo, o falso negativo do P05 mostra que as regras determinísticas também precisam ser validadas: uma regra simples demais (“preenchido é conforme”) introduziu um erro

que o modelo, no mesmo plano, não cometeu nos demais campos com texto provisório. A combinação mais robusta, apontada como trabalho futuro, é usar regras para detectar campos vazios ou provisórios, atribuindo-lhes AUSENTE de forma determinística, e reservar o modelo para julgar a qualidade dos campos preenchidos.

Do ponto de vista da gestão do curso, o sistema evidenciou um problema concreto e sistemático, a ausência do horário de atendimento em mais da metade dos planos, que pode ser tratado com uma orientação aos docentes no início do semestre. Esse tipo de diagnóstico agregado, difícil de obter em uma conferência manual plano a plano, ilustra o potencial da ferramenta para apoiar ações da coordenação e do NDE, em linha com as aplicações de IA na gestão educacional descritas por Sposato (2025).

É importante delimitar o alcance desses resultados. A concordância foi medida com um oráculo estrutural em um único critério e completada pela verificação dos apontamentos do sistema, o que permite estimar a precisão dos apontamentos, mas não o *recall* nos critérios interpretativos: um plano cuja metodologia fosse inadequada e tivesse sido considerada OK pelo sistema não seria detectado por esse procedimento. A estimativa completa exige a anotação de todos os critérios por avaliadores humanos, conforme o protocolo da Seção 3.6. Além disso, o corpus de 29 planos, de um único curso e semestre, limita a generalização dos números obtidos.

6 CONCLUSÃO

Este trabalho investigou de que forma um sistema baseado em modelos de linguagem pode apoiar a verificação da conformidade dos planos de ensino às normas do IFC, indicando de forma confiável os itens que exigem revisão e preservando o controle humano sobre a decisão final. Para responder a essa questão, foi desenvolvido e avaliado um sistema semiautomático que obtém os planos de ensino de uma pasta compartilhada, extrai e segmenta seu texto, avalia doze critérios com um LLM aberto executado localmente, combina essa avaliação com regras determinísticas e apresenta os resultados em um painel *web*.

Os objetivos específicos foram atendidos. Os elementos obrigatórios do plano de ensino foram convertidos em doze critérios com regras explícitas, definidos a partir da Organização Didática do IFC e das orientações do orientador e ajustados ao longo das iterações (primeiro objetivo). O *pipeline* de automação foi implementado no n8n, com extração e segmentação de texto, saída estruturada por esquema JSON, regras determinísticas e registro rastreável de cada validação (segundo objetivo). O *dashboard* permite acompanhar os indicadores do lote, consultar o status e a justificativa de cada critério e abrir o PDF original (terceiro objetivo). Por fim, o sistema foi avaliado com os 29 planos de ensino do semestre 2024/1 quanto à robustez, ao desempenho e à concordância, e a análise de erros resultou em propostas concretas de aprimoramento (quarto objetivo).

Os resultados mostram que a abordagem é viável. Ao longo de cinco iterações, a proporção de planos com falha técnica caiu de 86,2% para zero, e a versão final processou o lote completo em 8 minutos e 54 segundos, com 11,5 segundos de inferência por plano em uma GPU de 6 GB, sem custos por requisição e sem envio dos documentos a serviços externos. No critério verificável por oráculo estrutural, o sistema alcançou *recall* de 100%, precisão de 93,8% e Kappa de Cohen de 0,93, e todos os apontamentos verificados nos demais critérios correspondiam a não conformidades reais. A validação revelou ainda um problema sistemático no curso, a ausência do horário de atendimento em mais da metade dos planos, que dificilmente seria percebido em uma conferência manual plano a plano. Esses resultados indicam que o sistema pode direcionar a atenção da coordenação e do NDE para os planos e os itens que exigem revisão. Se essa triagem reduz, de fato, o tempo e o esforço da conferência é uma questão que este trabalho não mediu e que depende da avaliação com usuários indicada nas limitações.

A principal lição do trabalho é que a confiabilidade de um sistema com LLMs depende menos do modelo do que da engenharia em torno dele. As falhas mais graves das primeiras versões, como respostas truncadas, JSON malformado e *prompts* cortados pelo limite de contexto, foram resolvidas com controle de concorrência, saída estruturada e dimensionamento do contexto, e não com a troca do modelo. Da mesma forma, atribuir as verificações exatas a regras determinísticas e os julgamentos interpretativos ao modelo mostrou-se uma divisão de trabalho eficaz, desde que as próprias regras também sejam validadas, como evidenciou o falso negativo causado pela aceitação de um texto provisório do SIGAA.

6.1 LIMITAÇÕES

Os resultados devem ser interpretados à luz de algumas limitações. A primeira é a ausência de uma referência humana completa: a concordância foi medida com um oráculo estrutural em um único critério e complementada pela verificação dos apontamentos do sistema, o que permite avaliar a precisão dos apontamentos, mas não estimar os falsos negativos nos critérios interpretativos. A anotação de todos os critérios por membros do NDE, prevista no protocolo da Seção 3.6 e para a qual foram preparados os artefatos do Apêndice C, não coube no período deste trabalho. A segunda é o tamanho e a homogeneidade do corpus: 29 planos de um único curso, de um único semestre e de um único sistema de origem. A terceira é a dependência do formato do SIGAA, pois a segmentação se baseia nos rótulos de suas seções e precisaria ser revista caso o modelo do documento mude. A quarta é a instabilidade do modelo na escolha entre INCOMPLETO e AUSENTE, que afeta o status geral de alguns planos. Por fim, não foi realizada uma avaliação com usuários sobre a utilidade percebida do *dashboard* e o impacto real na carga de trabalho da coordenação.

6.2 TRABALHOS FUTUROS

A partir dos resultados e das limitações, sugerem-se como trabalhos futuros, em primeiro lugar, a avaliação com o NDE, executando o protocolo de anotação humana de todos os critérios e calculando, por critério, precisão, *recall*, F_1 e Kappa de Cohen, e ampliando o corpus para outros semestres e cursos. No plano técnico, a melhoria mais imediata é a detecção determinística de campos vazios e provisórios: atribuir AUSENTE por regra a seções vazias ou que contenham apenas textos provisórios do SIGAA, como “Não Cadastrado!” e “A cadastrar”, eliminaria o falso negativo identificado e a instabilidade entre INCOMPLETO e AUSENTE

nesses casos. Também se propõe gerar a observação geral por código, a partir dos status finais dos critérios, para eliminar as contradições observadas.

Em outra direção, a incorporação do PPC do curso e das ementas oficiais em uma base vetorial, por meio de RAG, permitiria verificar, por exemplo, se a ementa e as referências do plano correspondem às aprovadas no PPC. A comparação de modelos maiores ou mais recentes, locais e externos, sobre o mesmo corpus, mediria o ganho de qualidade frente ao custo e ao tempo de processamento. Por fim, do ponto de vista institucional, o sistema poderia notificar automaticamente os docentes sobre os itens pendentes e ser integrado ao momento de cadastro do plano no SIGAA, antes de sua publicação. Para isso, porém, seria preciso medir, com coordenadores, membros do NDE e docentes, a utilidade percebida, a clareza das justificativas e o tempo economizado em relação à conferência manual.

O código-fonte do sistema, os fluxos do n8n, os scripts de análise e os dados desta avaliação estão disponíveis no repositório `https://github.com/vslongo/vslongo-workflows`

REFERÊNCIAS

ADIGUZEL, Tufan; KAYA, Mehmet Haldun; CANSU, Fatih Kürşat. Revolutionizing education with AI: exploring the transformative potential of ChatGPT. **Contemporary Educational Technology**, [*S. l.*], v. 15, n. 3, art. ep429, 2023. DOI: https://doi.org/10.30935/cedtech/13152. Disponível em: https://doi.org/10.30935/cedtech/13152. Acesso em: 17 set. 2026.

AMERSHI, Saleema *et al.* Guidelines for human-AI interaction. *In*: CONFERENCE ON HUMAN FACTORS IN COMPUTING SYSTEMS, 2019, Glasgow. **Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems**. New York: Association for Computing Machinery, 2019. p. 1–13. DOI: https://doi.org/10.1145/3290605.3300233. Disponível em: https://doi.org/10.1145/3290605.3300233. Acesso em: 19 out. 2026.

BECK, Kent *et al.* **Manifesto for agile software development**. [*S. l.*]: agilemanifesto.org, 2001. Disponível em: https://agilemanifesto.org/. Acesso em: 19 out. 2026.

BRASIL. **Lei nº 9.394, de 20 de dezembro de 1996**. Estabelece as diretrizes e bases da educação nacional. Brasília, DF: Presidência da República, 1996. Disponível em: https://www.planalto.gov.br/ccivil_03/leis/l9394.htm. Acesso em: 19 out. 2026.

____. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 19 out. 2026.

BROWN, Tom B. *et al.* Language models are few-shot learners. *In*: CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS, 34., 2020, [*S. l.*]. **Advances in Neural Information Processing Systems 33**. Red Hook: Curran Associates, 2020. p. 1877–1901. Disponível em: https://arxiv.org/abs/2005.14165. Acesso em: 19 out. 2026.

CHANG, Yupeng *et al.* A survey on evaluation of large language models. **ACM Transactions on Intelligent Systems and Technology**, New York, v. 15, n. 3, art. 39, 2024. DOI: https://doi.org/10.1145/3641289. Disponível em: https://doi.org/10.1145/3641289. Acesso em: 19 out. 2026.

CNN BRASIL. **Metade dos universitários brasileiros usa inteligência artificial, diz pesquisa**. São Paulo: CNN Brasil, 2023. Disponível em: https://www.cnnbrasil.com.br/tecnologia/metade-dos-universitarios-brasileiros-usa-inteligencia-artificial-diz-pesquisa/. Acesso em: 17 set. 2026.

COHEN, Jacob. A coefficient of agreement for nominal scales. **Educational and Psychological Measurement**, [*S. l.*], v. 20, n. 1, p. 37–46, 1960. DOI: https://doi.org/10.1177/001316446002000104. Disponível em: https://doi.org/10.1177/001316446002000104. Acesso em: 19 out. 2026.

COHN, Mike. **User stories applied: for agile software development**. Boston: Addison-Wesley, 2004.

DAO, Tri *et al.* FlashAttention: fast and memory-efficient exact attention with IO-awareness. *In*: CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS, 36., 2022, New Orleans. **Advances in Neural Information Processing Systems 35**. Red Hook: Curran Associates, 2022. Disponível em: https://arxiv.org/abs/2205.14135. Acesso em: 19 out. 2026.

DETTMERS, Tim *et al.* LLM.int8(): 8-bit matrix multiplication for transformers at scale. *In*: CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS, 36., 2022, New Orleans. **Advances in Neural Information Processing Systems 35**. Red Hook: Curran Associates, 2022. Disponível em: https://arxiv.org/abs/2208.07339. Acesso em: 19 out. 2026.

GAO, Yunfan *et al.* **Retrieval-augmented generation for large language models: a survey**. [*S. l.*]: arXiv, 2023. DOI: https://doi.org/10.48550/arXiv.2312.10997. Disponível em: https://doi.org/10.48550/arXiv.2312.10997. Acesso em: 19 out. 2026.

GGML-ORG. **llama.cpp: LLM inference in C/C++**. [*S. l.*]: GitHub, 2026. Disponível em: https://github.com/ggml-org/llama.cpp. Acesso em: 19 out. 2026.

GIL, Antonio Carlos. **Como elaborar projetos de pesquisa**. 6. ed. São Paulo: Atlas, 2017.

GILARDI, Fabrizio; ALIZADEH, Meysam; KUBLI, Maël. ChatGPT outperforms crowd workers for text-annotation tasks. **Proceedings of the National Academy of Sciences**, Washington, DC, v. 120, n. 30, art. e2305016120, 2023. DOI: https://doi.org/10.1073/pnas.2305016120. Disponível em: https://doi.org/10.1073/pnas.2305016120. Acesso em: 19 out. 2026.

HOARE, Joseph; SMITH-FERGUSON, Jules. Using AI to boost evidence-based teaching and learning: a collaborative approach across a network of schools. **Theory Into Practice**, [*S. l.*], v. 64, n. 4, p. 407–420, 2025. DOI: https://doi.org/10.1080/00405841.2025.2528547. Disponível em: https://doi.org/10.1080/00405841.2025.2528547. Acesso em: 17 set. 2026.

INSTITUTO FEDERAL CATARINENSE. **Organização Didática dos Cursos do IFC**. Blumenau: IFC, 2022. Disponível em: https://ensino.ifc.edu.br/wp-content/uploads/sites/37/2022/12/Organizacao-Didatica-dos-Cursos-do-IFC.pdf. Acesso em: 19 out. 2026.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010:2011: systems and software engineering: systems and software Quality Requirements and Evaluation (SQuaRE): system and software quality models**. Geneva: ISO, 2011. Disponível em: https://www.iso.org/standard/35733.html. Acesso em: 19 out. 2026.

JI, Ziwei *et al.* Survey of hallucination in natural language generation. **ACM Computing Surveys**, New York, v. 55, n. 12, art. 248, 2023. DOI: https://doi.org/10.1145/3571730. Disponível em: https://doi.org/10.1145/3571730. Acesso em: 19 out. 2026.

JSON SCHEMA. **JSON Schema: specification 2020-12**. [*S. l.*]: json-schema.org, 2022. Disponível em: https://json-schema.org/specification. Acesso em: 19 out. 2026.

JURAFSKY, Daniel; MARTIN, James H. **Speech and language processing: an introduction to natural language processing, computational linguistics, and speech recognition with language models**. Stanford: Stanford University, 2025. Rascunho on-line. Disponível em: https://web.stanford.edu/~{}jurafsky/slp3/. Acesso em: 19 out. 2026.

KASNECI, Enkelejda *et al.* ChatGPT for good? On opportunities and challenges of large language models for education. **Learning and Individual Differences**, [*S. l.*], v. 103, art. 102274, 2023. DOI: https://doi.org/10.1016/j.lindif.2023.102274. Disponível em: https://doi.org/10.1016/j.lindif.2023.102274. Acesso em: 19 out. 2026.

KREUZBERGER, Dominik; KüHL, Niklas; HIRSCHL, Sebastian. Machine Learning Operations (MLOps): overview, definition, and architecture. **IEEE Access**, Piscataway, v. 11,

p. 31866–31879, 2023. DOI: https://doi.org/10.1109/ACCESS.2023.3262138. Disponível em: https://doi.org/10.1109/ACCESS.2023.3262138. Acesso em: 17 set. 2026.

LANDIS, J. Richard; KOCH, Gary G. The measurement of observer agreement for categorical data. **Biometrics**, [*S. l.*], v. 33, n. 1, p. 159–174, 1977. DOI: https://doi.org/10.2307/2529310. Disponível em: https://doi.org/10.2307/2529310. Acesso em: 19 out. 2026.

LEWIS, Patrick *et al.* Retrieval-augmented generation for knowledge-intensive NLP tasks. *In*: CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS, 34., 2020, [*S. l.*]. **Advances in Neural Information Processing Systems 33**. Red Hook: Curran Associates, 2020. p. 9459–9474. Disponível em: https://arxiv.org/abs/2005.11401. Acesso em: 17 set. 2026.

LIBâNEO, José Carlos. **Didática**. São Paulo: Cortez, 1994.

LIU, Nelson F. *et al.* Lost in the middle: how language models use long contexts. **Transactions of the Association for Computational Linguistics**, Cambridge, v. 12, p. 157–173, 2024. DOI: https://doi.org/10.1162/tacl_a_00638. Disponível em: https://doi.org/10.1162/tacl_a_00638. Acesso em: 19 out. 2026.

MERKEL, Dirk. Docker: lightweight Linux containers for consistent development and deployment. **Linux Journal**, Houston, v. 2014, n. 239, art. 2, 2014. Disponível em: https://dl.acm.org/doi/10.5555/2600239.2600241. Acesso em: 19 out. 2026.

METLI, Akın. Articles on education and artificial intelligence: a bibliometric analysis. **Journal of Social Sciences and Education**, [*S. l.*], v. 6, n. Education Special Issue, p. 279–312, 2023. DOI: https://doi.org/10.53047/josse.1352197. Disponível em: https://doi.org/10.53047/josse.1352197. Acesso em: 17 set. 2026.

MOZILLA. **PDF.js**. Mountain View: Mozilla Foundation, 2026. Disponível em: https://mozilla.github.io/pdf.js/. Acesso em: 19 out. 2026.

N8N. **n8n Docs**. Berlin: n8n GmbH, 2026. Disponível em: https://docs.n8n.io/. Acesso em: 19 out. 2026.

OBJECT MANAGEMENT GROUP. **OMG Unified Modeling Language (OMG UML): version 2.5.1**. Needham: OMG, 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1/. Acesso em: 19 out. 2026.

OLLAMA. **Structured outputs**. [*S. l.*]: Ollama Blog, 6 dez. 2024. Disponível em: https://ollama.com/blog/structured-outputs. Acesso em: 19 out. 2026.

____. **Ollama: get up and running with large language models**. [*S. l.*]: GitHub, 2026. Disponível em: https://github.com/ollama/ollama. Acesso em: 19 out. 2026.

PEDRó, Francesc *et al.* **Artificial intelligence in education: challenges and opportunities for sustainable development**. Paris: UNESCO, 2019. Disponível em: https://unesdoc.unesco.org/ark:/48223/pf0000366994. Acesso em: 17 set. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016.

QWEN TEAM. **Qwen2.5 technical report**. [*S. l.*]: arXiv, 2024. DOI: https://doi.org/10.48550/arXiv.2412.15115. Disponível em: https://doi.org/10.48550/arXiv.2412.15115. Acesso em: 19 out. 2026.

RUSSELL, Stuart; NORVIG, Peter. **Artificial intelligence: a modern approach**. 4. ed. Hoboken: Pearson, 2021.

SANTOS, Genesí Borba Gomes Alves; SILVA, Luís Gustavo Alexandre da; LIMA, Daniela da Costa Britto Pereira. Plataformas digitais e inteligência artificial na gestão das redes públicas de ensino e das escolas: conquistas e desafios. **Jornal de Políticas Educacionais**, Curitiba, v. 19, n. 1, 2025. DOI: https://doi.org/10.5380/jpe.v19i1.98302. Disponível em: https://doi.org/10.5380/jpe.v19i1.98302. Acesso em: 17 set. 2026.

SHNEIDERMAN, Ben. **Human-centered AI**. Oxford: Oxford University Press, 2022.

SOKOLOVA, Marina; LAPALME, Guy. A systematic analysis of performance measures for classification tasks. **Information Processing & Management**, [*S. l.*], v. 45, n. 4, p. 427–437, 2009. DOI: https://doi.org/10.1016/j.ipm.2009.03.002. Disponível em: https://doi.org/10.1016/j.ipm.2009.03.002. Acesso em: 19 out. 2026.

SOMMERVILLE, Ian. **Software engineering**. 10. ed. Boston: Pearson, 2016.

SPOSATO, Martin. Artificial intelligence in educational leadership: a comprehensive taxonomy and future directions. **International Journal of Educational Technology in Higher Education**, [*S. l.*], v. 22, n. 1, art. 20, 2025. DOI: https://doi.org/10.1186/s41239-025-00517-1. Disponível em: https://doi.org/10.1186/s41239-025-00517-1. Acesso em: 17 set. 2026.

TAN, Xiao; CHENG, Gary; LING, Man Ho. Artificial intelligence in teaching and teacher professional development: a systematic review. **Computers and Education: Artificial Intelligence**, [*S. l.*], v. 8, art. 100355, 2025. DOI: https://doi.org/10.1016/j.caeai.2024.100355. Disponível em: https://doi.org/10.1016/j.caeai.2024.100355. Acesso em: 17 set. 2026.

UNITED STATES OF AMERICA. Department of Education. Office of Educational Technology. **Artificial intelligence and the future of teaching and learning: insights and recommendations**. Washington, DC: Office of Educational Technology, 2023. Disponível em: https://www.ed.gov/sites/ed/files/documents/ai-report/ai-report.pdf. Acesso em: 19 out. 2026.

VASWANI, Ashish *et al.* Attention is all you need. *In*: CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS, 31., 2017, Long Beach. **Advances in Neural Information Processing Systems 30**. Red Hook: Curran Associates, 2017. p. 5998–6008. Disponível em: https://arxiv.org/abs/1706.03762. Acesso em: 17 set. 2026.

Vieira Filho, Marcelo Muller. **Aplicação de PLN na recomendação de conteúdo educacional**. Monografia (Trabalho de Conclusão de Curso (Graduação em Sistemas de Informação)) — Universidade Federal de Santa Catarina, Florianópolis, 2024. Disponível em: https://repositorio.ufsc.br/handle/123456789/255701. Acesso em: 17 set. 2026.

VIEIRA, Luis Antonio Braga; MOURA, Mayra Camelo Madeira de. Inteligência artificial na educação profissional e tecnológica: experiências pedagógicas nos institutos federais. **Revista Brasileira da Educação Profissional e Tecnológica**, Natal, v. 1, n. 25, art. e15906, 2025. DOI: https://doi.org/10.15628/rbept.2025.15906. Disponível em: https://doi.org/10.15628/rbept.2025.15906. Acesso em: 17 set. 2026.

WALTER, Yoshija. Embracing the future of artificial intelligence in the classroom: the relevance of AI literacy, prompt engineering, and critical thinking in modern education. **International Journal of Educational Technology in Higher Education**, [*S. l.*], v. 21, n. 1, art. 15, 2024. DOI: https://doi.org/10.1186/s41239-024-00448-3. Disponível em: https://doi.org/10.1186/s41239-024-00448-3. Acesso em: 17 set. 2026.

WHITE, Jules *et al.* **A prompt pattern catalog to enhance prompt engineering with ChatGPT**. [*S. l.*]: arXiv, 2023. DOI: https://doi.org/10.48550/arXiv.2302.11382. Disponível em: https://doi.org/10.48550/arXiv.2302.11382. Acesso em: 19 out. 2026.

WILLARD, Brandon T.; LOUF, Rémi. **Efficient guided generation for large language models**. [*S. l.*]: arXiv, 2023. DOI: https://doi.org/10.48550/arXiv.2307.09702. Disponível em: https://doi.org/10.48550/arXiv.2307.09702. Acesso em: 19 out. 2026.

YIN, Robert K. **Estudo de caso: planejamento e métodos**. 5. ed. Porto Alegre: Bookman, 2015.

ZHAO, Wayne Xin *et al.* **A survey of large language models**. [*S. l.*]: arXiv, 2023. DOI: https://doi.org/10.48550/arXiv.2303.18223. Disponível em: https://doi.org/10.48550/arXiv.2303.18223. Acesso em: 19 out. 2026.

ZHENG, Lianmin *et al.* Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *In*: CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS, 37., 2023, New Orleans. **Advances in Neural Information Processing Systems 36**. Red Hook: Curran Associates, 2023. Disponível em: https://arxiv.org/abs/2306.05685. Acesso em: 19 out. 2026.

APÊNDICE A – *PROMPT* E ESQUEMA JSON ENVIADOS AO MODELO

Este apêndice reproduz o código do nó *Preparar Prompt* do fluxo de validação, que monta, para cada plano, o *prompt* enviado ao modelo de linguagem, o esquema JSON usado no parâmetro `format` do Ollama e o *payload* alternativo para a API do Google Gemini. As expressões `${...}` são substituídas pelo texto de cada seção do plano no momento da execução.

```
return $input.all().map((item, index) => {
const sections = item.json;

// Recupera o nome do arquivo lido da pasta (mantido no lineage do item)
const fileName = $('Ler PDFs da Pasta').itemMatching(index)?.binary?.data?.fileName
    || 'arquivo_desconhecido';
// Link relativo: servido pelo webhook /webhook/pdf do workflow do dashboard
const fileLink = `pdf?arquivo=${encodeURIComponent(fileName)}`;

// Limites maiores para campos que precisam ser lidos por inteiro (ex.: contar refer
    ências)
const trunc = (str, max = 1500) => str ? str.substring(0, max) + (str.length > max ?
     '...' : '') : '(não preenchido)';

//  CONFIGURE O MODELO AQUI (ex: qwen2.5:7b, qwen2.5:14b, llama3.2, gemma2)
const OLLAMA_MODEL = 'qwen2.5:7b';

// Schema imposto ao Ollama via "format": garante os campos e enums exatos na
    resposta
const campos = ['identificacao','ementa','metodologia','procedimentos_avaliacao','
    objetivos_geral','objetivos_especificos','conteudo_programatico','cronograma','
    avaliacoes','referencias','horario_atendimento','coerencia'];
const campoSchema = {
  type: 'object',
  properties: {
    status: { type: 'string', enum: ['OK', 'INCOMPLETO', 'AUSENTE'] },
    obs: { type: 'string' }
  },
  required: ['status', 'obs']
};
const formatSchema = {
  type: 'object',
  properties: Object.fromEntries(campos.map(c => [c, campoSchema])),
  required: [...campos, 'status_geral', 'obs_geral']
};
formatSchema.properties.status_geral = { type: 'string', enum: ['OK', 'PENDENTE', '
    REPROVADO'] };
```

```
formatSchema.properties.obs_geral = { type: 'string' };

const prompt = `Você é um avaliador de planos de ensino de uma instituição de ensino superior brasileira.
Avalie cada campo APENAS com base no texto fornecido abaixo. Não invente informações.
Retorne APENAS um JSON válido, sem texto adicional, sem markdown.
Responda SEMPRE em português brasileiro (pt-BR), inclusive todas as observações.

Status possíveis por campo:
- "AUSENTE": o texto do campo está vazio, é "(não preenchido)" ou não tem nenhuma relação com o campo
- "INCOMPLETO": há conteúdo relacionado, mas não atende às regras do campo
- "OK": o conteúdo atende às regras do campo
IMPORTANTE: se houver texto real e relacionado no campo, NUNCA use AUSENTE -- escolha entre OK e INCOMPLETO.

Regras por campo:
- identificacao: OK se contiver nome da instituição, nome do componente curricular (disciplina) e nome do docente. Se faltar qualquer um dos três, INCOMPLETO e diga qual falta.
- ementa: OK se descrever os temas da disciplina em texto corrido com pelo menos 15 palavras.
- objetivos_geral: OK se houver qualquer texto preenchido.
- objetivos_especificos: OK se houver qualquer texto preenchido.
- conteudo_programatico: OK se listar os tópicos ou unidades de conteúdo a serem trabalhados.
- metodologia: OK se descrever COMO as aulas serão conduzidas (ex.: aulas expositivas, práticas, projetos, recursos didáticos).
- procedimentos_avaliacao: OK se descrever os instrumentos de avaliação. NÃO exija menção a interdisciplinaridade.
- cronograma: OK se apresentar datas, semanas ou qualquer distribuição temporal dos conteúdos.
- avaliacoes: OK se indicar as avaliações previstas (instrumentos, datas ou pesos).
- horario_atendimento: OK se informar dia(s) e/ou horário(s) de atendimento ao aluno.
- referencias: NÃO conte nada. Use a CONTAGEM VERIFICADA informada nos dados abaixo. OK somente se houver no mínimo 3 básicas E 5 complementares; caso contrário INCOMPLETO, informando na obs quantas há de cada tipo.
- coerencia: OK se os objetivos, a metodologia e os procedimentos de avaliação forem compatíveis entre si; aponte na obs qualquer contradição encontrada.

Status geral:
- "OK": todos os campos estão OK
- "PENDENTE": um ou mais campos estão INCOMPLETO (e nenhum AUSENTE)
- "REPROVADO": um ou mais campos estão AUSENTE

```

```
Formato de retorno (JSON puro):
{
  "identificacao": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação curta"},
  "ementa": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação curta"},
  "metodologia": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação curta"},
  "procedimentos_avaliacao": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação
    curta"},
  "objetivos_geral": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação curta"},
  "objetivos_especificos": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação
    curta"},
  "conteudo_programatico": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação
    curta"},
  "cronograma": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação curta"},
  "avaliacoes": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação curta"},
  "referencias": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação curta"},
  "horario_atendimento": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação
    curta"},
  "coerencia": {"status": "OK|INCOMPLETO|AUSENTE", "obs": "observação curta"},
  "status_geral": "OK|PENDENTE|REPROVADO",
  "obs_geral": "resumo geral do plano em 1 frase"
}

DADOS DO PLANO DE ENSINO:

IDENTIFICAÇÃO (início do documento):
${trunc(sections.identificacao, 800)}

EMENTA:
${trunc(sections.ementa)}

METODOLOGIA:
${trunc(sections.metodologia)}

PROCEDIMENTOS DE AVALIAÇÃO:
${trunc(sections.procedimentos_avaliacao)}

OBJETIVOS GERAL:
${trunc(sections.objetivos_geral)}

OBJETIVOS ESPECÍFICOS:
${trunc(sections.objetivos_especificos)}

CONTEÚDO PROGRAMÁTICO:
${trunc(sections.conteudo_programatico, 2500)}

CRONOGRAMA:
${trunc(sections.cronograma, 2500)}
```

```
106
107 AVALIAÇÕES:
108 ${trunc(sections.avaliacoes)}
109
110 HORÁRIO DE ATENDIMENTO:
111 ${trunc(sections.horario_atendimento, 500)}
112
113 REFERÊNCIAS (CONTAGEM VERIFICADA -- use exatamente estes números):
114 ${sections.ref_basicas} referência(s) básica(s) e ${sections.ref_complementares}
        complementar(es).`;
115
116 return {
117     json: {
118       fileName,
119       fileLink,
120       refBasicas: sections.ref_basicas ?? 0,
121       refComplementares: sections.ref_complementares ?? 0,
122       temObjGeral: !!(sections.objetivos_geral || '').trim(),
123       temObjEspecificos: !!(sections.objetivos_especificos || '').trim(),
124       ollamaPayload: JSON.stringify({
125         model: OLLAMA_MODEL,
126         prompt,
127         stream: false,
128         format: formatSchema,
129         keep_alive: '30m',
130         // num_ctx 8192: maior prompt medido ~4,4k tokens (+ resposta ~300); 12288 n
        ão cabia na GPU de 6 GB
131         options: { num_ctx: 8192, num_predict: 1500, temperature: 0.1 }
132       }),
133       // Payload alternativo para o Google Gemini (ver README, seção "Usar o Gemini
        ")
134       geminiPayload: JSON.stringify({
135         contents: [{ parts: [{ text: prompt }] }],
136         generationConfig: { temperature: 0.1, responseMimeType: 'application/json' }
137       })
138     }
139   };
140 });
```

APÊNDICE B – SEGMENTAÇÃO DO TEXTO E INTERPRETAÇÃO DA RESPOSTA

O trecho a seguir mostra uma parte do texto extraído de um dos planos do corpus (P05) pelo nó *Extract from File*. Nele se observam as características discutidas no Capítulo 4: as quebras de linha da diagramação, o campo de horário de atendimento vazio, imediatamente seguido pela seção “Programa”, e os textos provisórios “Não Cadastrado!” do SIGAA.

```
Horário de Atendimento:
Programa
Objetivos: Não Cadastrado!
Conteúdo Programático: Não Cadastrado!
Cronograma de Aulas - Total de aulas da Turma: 72
Início Fim Descrição
16/02/2024 13/03/2024 Integrais, derivadas parciais e suas aplicações
```

O código do nó *Separar Texto por Tópicos*, que segmenta o texto nas seções do plano e conta as referências bibliográficas, é reproduzido a seguir.

```
// ponytail: contagem de referencias feita em codigo -- o LLM erra ao contar linhas de tabela.
// Ajuste TIPOS_MATERIAL se o PDF usar outros rotulos na coluna "Tipo de Material".
const TIPOS_MATERIAL = /^[ \t]*(Livro|Peri[oó]dico|Revista|Artigo|Outros|Site|Manual|Apostila|Norma|Disserta[cç][aã]o|Tese|V[ií]deo|Software|Anais|Cap[ií]tulo|E-?book)\s+\S/gim;
function contarRefs(bloco) {
  return ((bloco || '').match(TIPOS_MATERIAL) || []).length;
}
// start = início do cabeçalho (onde a seção anterior termina), position = início do conteúdo
function findSection(text, pattern) {
  const match = new RegExp(pattern, 'im').exec(text);
  return match ? { start: match.index, position: match.index + match[0].length } : null;
}

// Cabeçalhos ancorados no início da linha (template SIGAA); tolerantes a caixa e à falta de acentos.
// Sem a âncora, "As avaliações..." no meio do texto era tomado como a seção Avaliações.
const sectionDefs = [
  { key: 'ementa', pattern: '^Ementa\\s*:?' },
  { key: 'metodologia', pattern: '^Metodologia\\s*:' },
  { key: 'procedimentos_avaliacao', pattern: '^Procedimentos\\s+de\\s+Avalia[çc][ãa]o(\\s+da\\s+Aprendizagem)?\\s*:?' },
```

```
  { key: 'horario_atendimento', pattern: '^Hor[áa]rio\\s+de\\s+Atendimento\\s*:?' },
  { key: 'programa', pattern: '^Programa[ \\t]*$' }, // só marca o fim do horário de atendimento
  { key: 'objetivos', pattern: '^Objetivos?(\\s+Gerais?)?\\s*:' },
  { key: 'conteudo_programatico', pattern: '^Conte[úu]do\\s+Program[áa]tico\\s*:?' },
  { key: 'cronograma', pattern: '^Cronograma' },
  { key: 'avaliacoes', pattern: '^Avalia[çc][õo]es\\s*:?[ \\t]*$' },
  { key: 'referencias', pattern: '^Refer[êe]ncias(\\s+B[áa]sicas|\\s+Bibliogr[áa]ficas)?\\s*:?' }
];

return $input.all().map(item => {
  const text = item.json.text || '';

  const sectionPositions = sectionDefs
    .map(s => {
      const found = findSection(text, s.pattern);
      return found ? { key: s.key, ...found } : null;
    })
    .filter(Boolean)
    .sort((a, b) => a.position - b.position);

  const result = {};
  result.identificacao = text.substring(0, 500).trim();

  for (let i = 0; i < sectionPositions.length; i++) {
    const cur = sectionPositions[i];
    const next = sectionPositions[i + 1];
    const content = text.substring(cur.position, next ? next.start : text.length).trim();

    if (cur.key === 'objetivos') {
      // Aceita "GERAL:", "Objetivo Geral", "ESPECÍFICOS:", "Objetivos Específicos" etc.
      const geralMatch = /(OBJETIVO\s+)?GERAL\s*:?/i.exec(content);
      const espMatch = /(OBJETIVOS?\s+)?ESPEC[ÍI]FICOS?\s*:?/i.exec(content);
      const geralEnd = espMatch ? espMatch.index : content.length;
      result.objetivos_geral = geralMatch
        ? content.substring(geralMatch.index + geralMatch[0].length, geralEnd).trim()
        : (espMatch ? content.substring(0, espMatch.index).trim() : content);
      result.objetivos_especificos = espMatch
        ? content.substring(espMatch.index + espMatch[0].length).trim()
        : '';
    } else {
      result[cur.key] = content;
```

```
60      }
61    }
62
63    const iBas = text.search(/Refer[êe]ncias\s+B[áa]sicas/i);
64    const iCom = text.search(/Refer[êe]ncias\s+Complementares/i);
65    result.ref_basicas = iBas >= 0 ? contarRefs(text.slice(iBas, iCom > iBas ? iCom :
        text.length)) : 0;
66    result.ref_complementares = iCom >= 0 ? contarRefs(text.slice(iCom)) : 0;
67
68    return { json: result };
69  });
```

O código do nó *Parsear Validação*, que interpreta a resposta do modelo, aplica as regras determinísticas e monta o registro gravado na pasta de resultados, é reproduzido a seguir.

```
1   const CAMPOS = ['identificacao','ementa','metodologia','procedimentos_avaliacao','
        objetivos_geral','objetivos_especificos','conteudo_programatico','cronograma','
        avaliacoes','referencias','horario_atendimento','coerencia'];
2
3   function extrairJson(rawText) {
4     const m = (rawText || '').match(/\{[\s\S]*\}/);
5     const v = JSON.parse(m ? m[0] : rawText);
6     if (!v.status_geral) throw new Error('JSON sem status_geral');
7     return v;
8   }
9
10  const out = [];
11  const items = $input.all();
12
13  for (let index = 0; index < items.length; index++) {
14    const item = items[index];
15    const prep = $('Preparar Prompt').itemMatching(index)?.json || {};
16    const fileName = prep.fileName || 'arquivo_desconhecido';
17    const fileLink = prep.fileLink || '';
18
19    let validacao = null;
20    let ultimoErro = '';
21
22    // Erro reportado pelo nó HTTP (timeout, conexão etc.) -- mostra a causa real na
        planilha
23    const errHttp = item.json.error?.message || (typeof item.json.error === 'string' ?
        item.json.error : '');
24
25    try {
26      // Ollama: response.response · Gemini: candidates[0].content.parts[0].text
27      const rawText = item.json.response
28        || item.json.candidates?.[0]?.content?.parts?.[0]?.text
```

```
29      || '';
30    validacao = extrairJson(rawText);
31  } catch (e) {
32    ultimoErro = errHttp || e.message;
33  }
34
35  if (!validacao) {
36    validacao = {
37      status_geral: 'ERRO',
38      obs_geral: 'Falha ao processar resposta da IA: ' + ultimoErro
39    };
40  }
41
42  // ponytail: contagem deterministica do node "Separar Texto por Topicos" vence a
      da IA
43  if (validacao.status_geral !== 'ERRO') {
44    const nb = prep.refBasicas ?? 0;
45    const nc = prep.refComplementares ?? 0;
46    const okRef = nb >= 3 && nc >= 5;
47    validacao.referencias = {
48      status: okRef ? 'OK' : 'INCOMPLETO',
49      obs: okRef
50        ? `${nb} referências básicas e ${nc} complementares.`
51        : `${nb} referências básicas (mín. 3) e ${nc} complementares (mín. 5).`
52    };
53    // ponytail: objetivo preenchido = OK, qualquer que seja a redacao
54    if (prep.temObjGeral) validacao.objetivos_geral = { status: 'OK', obs: 'Objetivo
       geral preenchido.' };
55    if (prep.temObjEspecificos) validacao.objetivos_especificos = { status: 'OK',
      obs: 'Objetivos especificos preenchidos.' };
56
57    // ponytail: interdisciplinaridade nao reprova -- rede de seguranca caso o
      modelo ainda cobre
58    if (validacao.procedimentos_avaliacao?.status === 'INCOMPLETO' && /
      interdisciplinar/i.test(validacao.procedimentos_avaliacao.obs || '')) {
59      validacao.procedimentos_avaliacao = { status: 'OK', obs: 'Instrumentos de
      avaliacao descritos.' };
60    }
61
62    const sts = CAMPOS.map(f => validacao[f]?.status);
63    validacao.status_geral = sts.includes('AUSENTE') ? 'REPROVADO'
64      : sts.includes('INCOMPLETO') ? 'PENDENTE' : 'OK';
65  }
66
67  const s = (field) => validacao[field]?.status || 'ERRO';
68  const o = (field) => validacao[field]?.obs || '';
69
```

```
  out.push({
    json: {
      'Arquivo': fileName,
      'Link PDF': fileLink,
      'Data Validação': new Date().toLocaleString('pt-BR'),
      'Status Geral': validacao.status_geral || 'ERRO',
      'Observação Geral': validacao.obs_geral || '',
      'Ementa': s('ementa'),
      'Ementa Obs': o('ementa'),
      'Metodologia': s('metodologia'),
      'Metodologia Obs': o('metodologia'),
      'Proc. Avaliação': s('procedimentos_avaliacao'),
      'Proc. Avaliação Obs': o('procedimentos_avaliacao'),
      'Objetivos Geral': s('objetivos_geral'),
      'Objetivos Geral Obs': o('objetivos_geral'),
      'Objetivos Específicos': s('objetivos_especificos'),
      'Objetivos Específicos Obs': o('objetivos_especificos'),
      'Conteúdo Programático': s('conteudo_programatico'),
      'Conteúdo Programático Obs': o('conteudo_programatico'),
      'Cronograma': s('cronograma'),
      'Cronograma Obs': o('cronograma'),
      'Avaliações': s('avaliacoes'),
      'Avaliações Obs': o('avaliacoes'),
      'Referências': s('referencias'),
      'Referências Obs': o('referencias'),
      'Identificação': s('identificacao'),
      'Identificação Obs': o('identificacao'),
      'Horário Atendimento': s('horario_atendimento'),
      'Horário Atendimento Obs': o('horario_atendimento'),
      'Coerência': s('coerencia'),
      'Coerência Obs': o('coerencia')
    }
  });
  // Um JSON por plano em /data/resultados: reprocessar o mesmo PDF sobrescreve, sem
     duplicar
  const linha = out[out.length - 1];
  linha.binary = { data: {
    data: Buffer.from(JSON.stringify(linha.json, null, 2)).toString('base64'),
    mimeType: 'application/json',
    fileName: fileName.replace(/\.pdf$/i, '') + '.json'
  } };
}

return out;
```

APÊNDICE C – ARTEFATOS PARA A ANÁLISE DOS RESULTADOS

Os dados e *scripts* usados no Capítulo 5 acompanham este trabalho no repositório do projeto, na pasta `tcc-final/dados`:

- `validacoes_2026-10-19.csv`: resultado anonimizado da execução final, com uma linha por plano, os status dos doze critérios, a contagem de referências, os metadados de inferência e o indicador do oráculo estrutural do horário de atendimento;
- `anotacao_humana_modelo.csv`: planilha de anotação com uma linha por par (plano, critério), contendo o status atribuído pelo sistema e colunas em branco para o parecer humano (OK, INCOMPLETO ou AUSENTE) e para comentários;
- `analise.py`: *script* que calcula as estatísticas descritivas, a concordância com o oráculo estrutural e, quando recebe a planilha de anotação preenchida, as métricas da Seção 2.7 por critério e no total.

Para executar o protocolo de comparação com o NDE, recomenda-se ocultar a coluna `sistema` da planilha de anotação antes de distribuí-la aos avaliadores, para que o parecer humano não seja influenciado pela saída do sistema. Após a consolidação dos pareceres, as métricas são obtidas com o comando `python analise.py anotacao_humana.csv`. O código do *script* é reproduzido a seguir.

```
"""Métricas do capítulo de resultados, a partir de validacoes_2026-10-19.csv.

Uso: python analise.py [anotacao_humana.csv]

Sem argumento, calcula as estatísticas descritivas e a concordância do critério
"Horário de Atendimento" com o oráculo estrutural (texto vazio entre o cabeçalho
e a seção seguinte). Com a planilha de anotação humana preenchida (mesmo formato
de anotacao_humana_modelo.csv), calcula precisão, recall, F1, acurácia e Kappa
de Cohen por critério, tratando INCOMPLETO e AUSENTE como "não conforme".
"""
import csv
import statistics as st
import sys
from collections import Counter
from pathlib import Path

AQUI = Path(__file__).parent
CAMPOS = ['Identificação', 'Ementa', 'Objetivos Geral', 'Objetivos Específicos',
          'Conteúdo Programático', 'Metodologia', 'Proc. Avaliação', 'Cronograma',
```

```
20          'Avaliações', 'Horário Atendimento', 'Referências', 'Coerência']
21
22
23  def metricas(pares):
24      """pares = [(referencia, sistema)] com True = não conforme (classe positiva)."""
25      vp = sum(r and s for r, s in pares)
26      fp = sum((not r) and s for r, s in pares)
27      fn = sum(r and not s for r, s in pares)
28      vn = sum((not r) and not s for r, s in pares)
29      n = len(pares)
30      prec = vp / (vp + fp) if vp + fp else float('nan')
31      rec = vp / (vp + fn) if vp + fn else float('nan')
32      f1 = 2 * prec * rec / (prec + rec) if vp else 0.0
33      po = (vp + vn) / n
34      pe = ((vp + fp) * (vp + fn) + (fn + vn) * (fp + vn)) / n ** 2
35      kappa = (po - pe) / (1 - pe) if pe < 1 else 1.0
36      return dict(VP=vp, FP=fp, FN=fn, VN=vn, acuracia=po, precisao=prec, recall=rec,
        f1=f1, kappa=kappa)
37
38
39  def main():
40      linhas = list(csv.DictReader(open(AQUI / 'validacoes_2026-10-19.csv', encoding='
        utf-8')))
41      print(f'Planos: {len(linhas)}')
42      print('Status geral:', dict(Counter(l['status_geral'] for l in linhas)))
43      for c in CAMPOS:
44          print(f'  {c:24s}', dict(Counter(l[c] for l in linhas)))
45
46      for k in ['paginas', 'caracteres', 'tokens_prompt', 'tokens_resposta', 's_prompt
        ', 's_geracao']:
47          v = [float(l[k]) for l in linhas]
48          print(f'{k:16s} min={min(v):.2f} max={max(v):.2f} media={st.mean(v):.2f} '
49                f'mediana={st.median(v):.2f} dp={st.stdev(v):.2f}')
50      tps = [float(l['tokens_resposta']) / float(l['s_geracao']) for l in linhas]
51      print(f'Velocidade de geração: {st.mean(tps):.1f} tokens/s')
52
53      pares = [(l['horario_texto_vazio'] == '1', l['Horário Atendimento'] != 'OK') for
         l in linhas]
54      print('Horário x oráculo estrutural:', {k: round(v, 4) for k, v in metricas(
        pares).items()})
55
56      if len(sys.argv) > 1:
57          anot = {(a['plano'], a['criterio']): a['humano'].strip().upper()
58                  for a in csv.DictReader(open(sys.argv[1], encoding='utf-8')) if a['
        humano'].strip()}
59          todos = []
60          for c in CAMPOS:
```

```
61              p = [(anot[(l['plano'], c)] != 'OK', l[c] != 'OK') for l in linhas if (l
    ['plano'], c) in anot]
62              todos += p
63              if p:
64                  print(c, {k: round(v, 3) for k, v in metricas(p).items()})
65          if todos:
66              print('GERAL', {k: round(v, 3) for k, v in metricas(todos).items()})
67
68
69  if __name__ == '__main__':
70      main()
```